



**Введение в двойственные методы
оптимизации. Метод двойственного
градиентного подъёма. Метод
модифицированной функции Лагранжа.
ADMM.**

Даня Меркулов

ФКН ВШЭ

Введение в двойственные методы

Зачем нам двойственная задача

Двойственные методы решают не только прямую задачу, но и **двойственную**: наряду с x они ищут множители Лагранжа — двойственные переменные. На практике двойственные задачи и переменные встречаются сами по себе, и эта лекция отвечает на три вопроса о них.

Зачем нам двойственная задача

Двойственные методы решают не только прямую задачу, но и **двойственную**: наряду с x они ищут множители Лагранжа — двойственные переменные. На практике двойственные задачи и переменные встречаются сами по себе, и эта лекция отвечает на три вопроса о них.

- **Где встречаются?** Распределение ресурсов, рынки и энергосети, метод опорных векторов, распределение мощности по каналам связи — во всех этих задачах двойственные переменные возникают естественно.

Зачем нам двойственная задача

Двойственные методы решают не только прямую задачу, но и **двойственную**: наряду с x они ищут множители Лагранжа — двойственные переменные. На практике двойственные задачи и переменные встречаются сами по себе, и эта лекция отвечает на три вопроса о них.

- **Где встречаются?** Распределение ресурсов, рынки и энергосети, метод опорных векторов, распределение мощности по каналам связи — во всех этих задачах двойственные переменные возникают естественно.
- **Что означают?** Множитель — это *цена* ограничения: на сколько изменится оптимум, если ограничение чуть сдвинуть. Это теньевая цена ресурса.

Зачем нам двойственная задача

Двойственные методы решают не только прямую задачу, но и **двойственную**: наряду с x они ищут множители Лагранжа — двойственные переменные. На практике двойственные задачи и переменные встречаются сами по себе, и эта лекция отвечает на три вопроса о них.

- **Где встречаются?** Распределение ресурсов, рынки и энергосети, метод опорных векторов, распределение мощности по каналам связи — во всех этих задачах двойственные переменные возникают естественно.
- **Что означают?** Множитель — это *цена* ограничения: на сколько изменится оптимум, если ограничение чуть сдвинуть. Это теневая цена ресурса.
- **Зачем искать?** Иногда сами цены и есть искомый ответ; иногда двойственная задача проще прямой (вогнута, распадается на части); и она всегда даёт нижнюю оценку и критерий остановки.

Зачем нам двойственная задача

Двойственные методы решают не только прямую задачу, но и **двойственную**: наряду с x они ищут множители Лагранжа — двойственные переменные. На практике двойственные задачи и переменные встречаются сами по себе, и эта лекция отвечает на три вопроса о них.

- **Где встречаются?** Распределение ресурсов, рынки и энергосети, метод опорных векторов, распределение мощности по каналам связи — во всех этих задачах двойственные переменные возникают естественно.
- **Что означают?** Множитель — это *цена* ограничения: на сколько изменится оптимум, если ограничение чуть сдвинуть. Это теневая цена ресурса.
- **Зачем искать?** Иногда сами цены и есть искомый ответ; иногда двойственная задача проще прямой (вогнута, распадается на части); и она всегда даёт нижнюю оценку и критерий остановки.

Зачем нам двойственная задача

Двойственные методы решают не только прямую задачу, но и **двойственную**: наряду с x они ищут множители Лагранжа — двойственные переменные. На практике двойственные задачи и переменные встречаются сами по себе, и эта лекция отвечает на три вопроса о них.

- **Где встречаются?** Распределение ресурсов, рынки и энергосети, метод опорных векторов, распределение мощности по каналам связи — во всех этих задачах двойственные переменные возникают естественно.
- **Что означают?** Множитель — это *цена* ограничения: на сколько изменится оптимум, если ограничение чуть сдвинуть. Это теневая цена ресурса.
- **Зачем искать?** Иногда сами цены и есть искомый ответ; иногда двойственная задача проще прямой (вогнута, распадается на части); и она всегда даёт нижнюю оценку и критерий остановки.

Разберём по порядку: что такое множитель и что он значит, затем два примера, где двойственная переменная — это и есть ответ.

Напоминание: множитель Лагранжа и теневая цена

Коротко, чтобы зафиксировать обозначения (Лагранжиан и анализ чувствительности были в прошлых лекциях).

Напоминание: множитель Лагранжа и теневая цена

Коротко, чтобы зафиксировать обозначения (Лагранжиан и анализ чувствительности были в прошлых лекциях).

Для $\min_x f(x)$ при $Ax = b$ множитель λ есть цена за нарушение связи $Ax = b$:

$$L(x, \lambda) = f(x) + \lambda^\top (Ax - b).$$

Напоминание: множитель Лагранжа и теневая цена

Коротко, чтобы зафиксировать обозначения (Лагранжиан и анализ чувствительности были в прошлых лекциях).

Для $\min_x f(x)$ при $Ax = b$ множитель λ есть цена за нарушение связи $Ax = b$:

$$L(x, \lambda) = f(x) + \lambda^\top (Ax - b).$$

- При фиксированном λ получаем **безусловную** задачу $\min_x L(x, \lambda)$.

Напоминание: множитель Лагранжа и теневая цена

Коротко, чтобы зафиксировать обозначения (Лагранжиан и анализ чувствительности были в прошлых лекциях).

Для $\min_x f(x)$ при $Ax = b$ множитель λ есть цена за нарушение связи $Ax = b$:

$$L(x, \lambda) = f(x) + \lambda^\top (Ax - b).$$

- При фиксированном λ получаем **безусловную** задачу $\min_x L(x, \lambda)$.
- λ подбираем так, чтобы в оптимуме выполнялось $Ax = b \Rightarrow$ седловая точка $\max_\lambda \min_x L(x, \lambda)$.

Напоминание: множитель Лагранжа и теневая цена

Коротко, чтобы зафиксировать обозначения (Лагранжиан и анализ чувствительности были в прошлых лекциях).

Для $\min_x f(x)$ при $Ax = b$ множитель λ есть цена за нарушение связи $Ax = b$:

$$L(x, \lambda) = f(x) + \lambda^\top (Ax - b).$$

- При фиксированном λ получаем **безусловную** задачу $\min_x L(x, \lambda)$.
- λ подбираем так, чтобы в оптимуме выполнялось $Ax = b \Rightarrow$ седловая точка $\max_\lambda \min_x L(x, \lambda)$.

Напоминание: множитель Лагранжа и теневая цена

Коротко, чтобы зафиксировать обозначения (Лагранжиан и анализ чувствительности были в прошлых лекциях).

Для $\min_x f(x)$ при $Ax = b$ множитель λ есть цена за нарушение связи $Ax = b$:

$$L(x, \lambda) = f(x) + \lambda^\top (Ax - b).$$

- При фиксированном λ получаем **безусловную** задачу $\min_x L(x, \lambda)$.
- λ подбираем так, чтобы в оптимуме выполнялось $Ax = b \Rightarrow$ седловая точка $\max_\lambda \min_x L(x, \lambda)$.

Теневая цена (чувствительность): $\lambda^* = -\partial p^* / \partial b$ показывает, на сколько сдвинется оптимум $p^*(b)$ при сдвиге правой части b на единицу.

Зачем двойственные переменные: SVM

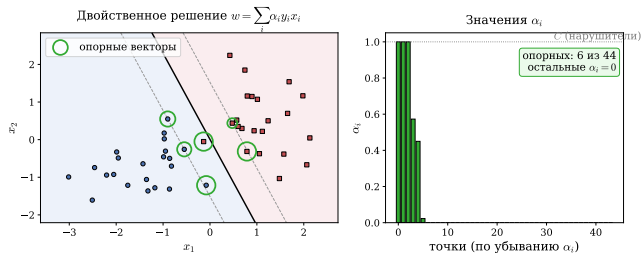


Figure 1: В SVM удобнее решать **двойственную** задачу: $\alpha_i > 0$ только у опорных векторов (здесь 6 из 44), поэтому решение разрежено.

У двойственной задачи есть два удобных свойства:

- активны лишь **опорные векторы**, решение разрежено;

Зачем двойственные переменные: SVM

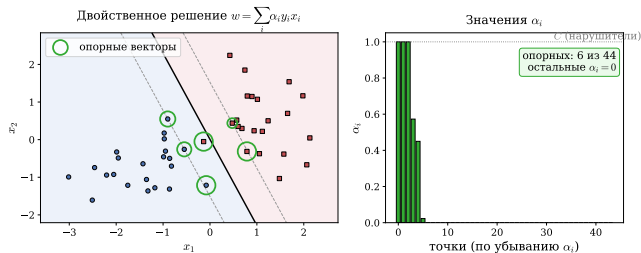


Figure 1: В SVM удобнее решать **двойственную** задачу: $\alpha_i > 0$ только у опорных векторов (здесь 6 из 44), поэтому решение разрежено.

У двойственной задачи есть два удобных свойства:

- активны лишь **опорные векторы**, решение разрежено;
- данные входят только через скалярные произведения $x_i^\top x_j$, что даёт переход к **ядрам**.

Зачем двойственные переменные: SVM

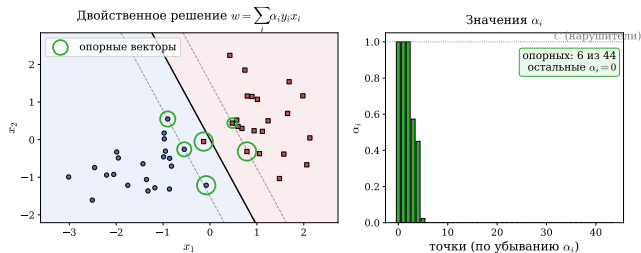


Figure 1: В SVM удобнее решать **двойственную** задачу: $\alpha_i > 0$ только у опорных векторов (здесь 6 из 44), поэтому решение разрежено.

У двойственной задачи есть два удобных свойства:

- активны лишь **опорные векторы**, решение разрежено;
- данные входят только через скалярные произведения $x_i^\top x_j$, что даёт переход к **ядрам**.

Зачем двойственные переменные: SVM

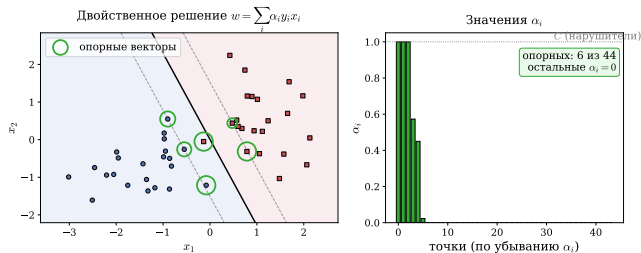


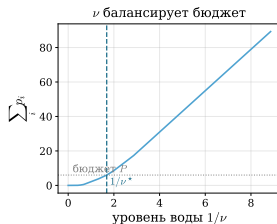
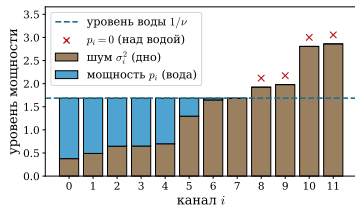
Figure 1: В SVM удобнее решать **двойственную** задачу: $\alpha_i > 0$ только у опорных векторов (здесь 6 из 44), поэтому решение разрежено.

У двойственной задачи есть два удобных свойства:

- активны лишь **опорные векторы**, решение разрежено;
- данные входят только через скалярные произведения $x_i^\top x_j$, что даёт переход к **ядрам**.

Откуда берётся эта двойственная задача (прямая $\rightarrow$ Лагранжиан $\rightarrow$ двойственная), разберём в конце лекции.

Зачем двойственные переменные: заполнение водой



Один скаляр, двойственная переменная ν (общий **уровень воды**), полностью задаёт оптимальное распределение по всем каналам. Классический пример, где множитель имеет прямой физический смысл. Полную постановку и вывод из условий ККТ разберём в конце лекции.

Figure 2: Распределение мощности по n каналам: ν есть единый **уровень воды**. Канал получает мощность $x_i = (1/\nu - 1/g_i)_+$, «дно» = $1/g_i$ (уровень шума); слабые каналы остаются «над водой» с нулевой мощностью.

Зачем решать двойственные задачи?

Прямая задача

$$\begin{aligned} & f_0(x) \rightarrow \min_{x \in \mathbb{R}^n} \\ \text{при} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_i(x) = 0, \quad i = 1, \dots, p \end{aligned}$$

Двойственная задача

$$\begin{aligned} & g(\lambda, \nu) = \min_{x \in \mathcal{D}} L(x, \lambda, \nu) = \\ \min_{x \in \mathcal{D}} & \left(f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^p \nu_i h_i(x) \right) \rightarrow \max_{\lambda \in \mathbb{R}^m, \nu \in \mathbb{R}^p} \\ & \text{при } \lambda \succeq 0 \end{aligned}$$

Зачем решать двойственные задачи?

Прямая задача

$$\begin{aligned} f_0(x) &\rightarrow \min_{x \in \mathbb{R}^n} \\ \text{при} \quad f_i(x) &\leq 0, \quad i = 1, \dots, m \\ h_i(x) &= 0, \quad i = 1, \dots, p \end{aligned}$$

Двойственная задача

$$\begin{aligned} g(\lambda, \nu) &= \min_{x \in \mathcal{D}} L(x, \lambda, \nu) = \\ \min_{x \in \mathcal{D}} \left(f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^p \nu_i h_i(x) \right) &\rightarrow \max_{\lambda \in \mathbb{R}^m, \nu \in \mathbb{R}^p} \\ &\text{при } \lambda \succeq 0 \end{aligned}$$

- **Теневые цены.** Двойственные переменные суть теневые цены ресурсов; они дают экономический смысл ограничениям.

Зачем решать двойственные задачи?

Прямая задача

$$\begin{aligned} f_0(x) &\rightarrow \min_{x \in \mathbb{R}^n} \\ \text{при } f_i(x) &\leq 0, \quad i = 1, \dots, m \\ h_i(x) &= 0, \quad i = 1, \dots, p \end{aligned}$$

Двойственная задача

$$\begin{aligned} g(\lambda, \nu) &= \min_{x \in \mathcal{D}} L(x, \lambda, \nu) = \\ \min_{x \in \mathcal{D}} \left(f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^p \nu_i h_i(x) \right) &\rightarrow \max_{\lambda \in \mathbb{R}^m, \nu \in \mathbb{R}^p} \\ &\text{при } \lambda \succeq 0 \end{aligned}$$

- **Теневые цены.** Двойственные переменные суть теневые цены ресурсов; они дают экономический смысл ограничениям.
- **Рыночное равновесие.** Двойственная задача часто описывает условия равновесия, что полезно в экономическом моделировании.

Зачем решать двойственные задачи?

Прямая задача

$$\begin{aligned} f_0(x) &\rightarrow \min_{x \in \mathbb{R}^n} \\ \text{при} \quad &f_i(x) \leq 0, \quad i = 1, \dots, m \\ &h_i(x) = 0, \quad i = 1, \dots, p \end{aligned}$$

Двойственная задача

$$\begin{aligned} g(\lambda, \nu) &= \min_{x \in \mathcal{D}} L(x, \lambda, \nu) = \\ \min_{x \in \mathcal{D}} \left(f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^p \nu_i h_i(x) \right) &\rightarrow \max_{\lambda \in \mathbb{R}^m, \nu \in \mathbb{R}^p} \\ &\text{при } \lambda \succeq 0 \end{aligned}$$

- **Теневые цены.** Двойственные переменные суть теневые цены ресурсов; они дают экономический смысл ограничениям.
- **Рыночное равновесие.** Двойственная задача часто описывает условия равновесия, что полезно в экономическом моделировании.
- **Нижние оценки.** $g(\lambda, \nu) \leq f_0(x^*)$ при любых $\lambda \succeq 0, \nu$, откуда оценка качества приближённых решений.

Зачем решать двойственные задачи?

Прямая задача

$$\begin{aligned} f_0(x) &\rightarrow \min_{x \in \mathbb{R}^n} \\ \text{при} \quad f_i(x) &\leq 0, \quad i = 1, \dots, m \\ h_i(x) &= 0, \quad i = 1, \dots, p \end{aligned}$$

Двойственная задача

$$\begin{aligned} g(\lambda, \nu) &= \min_{x \in \mathcal{D}} L(x, \lambda, \nu) = \\ \min_{x \in \mathcal{D}} \left(f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^p \nu_i h_i(x) \right) &\rightarrow \max_{\lambda \in \mathbb{R}^m, \nu \in \mathbb{R}^p} \\ &\text{при } \lambda \succeq 0 \end{aligned}$$

- **Теневые цены.** Двойственные переменные суть теневые цены ресурсов; они дают экономический смысл ограничениям.
- **Рыночное равновесие.** Двойственная задача часто описывает условия равновесия, что полезно в экономическом моделировании.
- **Нижние оценки.** $g(\lambda, \nu) \leq f_0(x^*)$ при любых $\lambda \succeq 0, \nu$, откуда оценка качества приближённых решений.
- **Зазор двойственности.** $f_0(x) - g(\lambda, \nu)$ измеряет неоптимальность; при выпуклости и условии Слейтера зазор равен нулю.

Сопряжённые функции

Двойственная функция через сопряжённую

Чтобы решать двойственную задачу $\max_{\nu} g(\nu)$, нужно уметь вычислять $g(\nu) = \min_x L(x, \nu)$. Для равенств $Ax = b$ этот минимум выражается через сопряжённую f^* . Выведем связь (множитель равенства обозначаем ν).

Двойственная функция через сопряжённую

Чтобы решать двойственную задачу $\max_{\nu} g(\nu)$, нужно уметь вычислять $g(\nu) = \min_x L(x, \nu)$. Для равенств $Ax = b$ этот минимум выражается через сопряжённую f^* . Выведем связь (множитель равенства обозначаем ν).

$g(\nu) = \min_x L(x, \nu)$, где для $Ax = b$:

$$L(x, \nu) = f(x) + \nu^T(Ax - b) = (f(x) + (A^T \nu)^T x) - b^T \nu$$

Двойственная функция через сопряжённую

Чтобы решать двойственную задачу $\max_{\nu} g(\nu)$, нужно уметь вычислять $g(\nu) = \min_x L(x, \nu)$. Для равенств $Ax = b$ этот минимум выражается через сопряжённую f^* . Выведем связь (множитель равенства обозначаем ν).

$g(\nu) = \min_x L(x, \nu)$, где для $Ax = b$:

$$L(x, \nu) = f(x) + \nu^T(Ax - b) = (f(x) + (A^T \nu)^T x) - b^T \nu$$

Минимизируем по x :

$$g(\nu) = \min_x [f(x) + (A^T \nu)^T x] - b^T \nu = -\max_x [(-A^T \nu)^T x - f(x)] - b^T \nu$$

Двойственная функция через сопряжённую

Чтобы решать двойственную задачу $\max_{\nu} g(\nu)$, нужно уметь вычислять $g(\nu) = \min_x L(x, \nu)$. Для равенств $Ax = b$ этот минимум выражается через сопряжённую f^* . Выведем связь (множитель равенства обозначаем ν).

$g(\nu) = \min_x L(x, \nu)$, где для $Ax = b$:

$$L(x, \nu) = f(x) + \nu^T(Ax - b) = (f(x) + (A^T \nu)^T x) - b^T \nu$$

Минимизируем по x :

$$g(\nu) = \min_x [f(x) + (A^T \nu)^T x] - b^T \nu = -\max_x [(-A^T \nu)^T x - f(x)] - b^T \nu$$

Выражение в скобках есть **определение сопряжённой**:

$$g(\nu) = -f^*(-A^T \nu) - b^T \nu$$

Двойственная функция через сопряжённую

Чтобы решать двойственную задачу $\max_{\nu} g(\nu)$, нужно уметь вычислять $g(\nu) = \min_x L(x, \nu)$. Для равенств $Ax = b$ этот минимум выражается через сопряжённую f^* . Выведем связь (множитель равенства обозначаем ν).

$g(\nu) = \min_x L(x, \nu)$, где для $Ax = b$:

$$L(x, \nu) = f(x) + \nu^T(Ax - b) = (f(x) + (A^T \nu)^T x) - b^T \nu$$

Минимизируем по x :

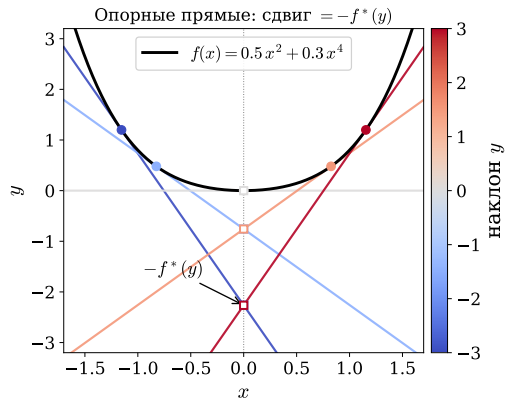
$$g(\nu) = \min_x [f(x) + (A^T \nu)^T x] - b^T \nu = -\max_x [(-A^T \nu)^T x - f(x)] - b^T \nu$$

Выражение в скобках есть **определение сопряжённой**:

$$g(\nu) = -f^*(-A^T \nu) - b^T \nu$$

Двойственная функция — это f^* с линейной подстановкой аргумента: вычисление двойственной сводится к вычислению сопряжённой. Поэтому скорость двойственного подъёма определяется свойствами f^* , которые мы и изучаем дальше. Через три слайда увидим: если f не сильно выпукла, то f^* негладкая и подъём сходится медленно.

Сопряжённая функция f^* : определение и геометрия

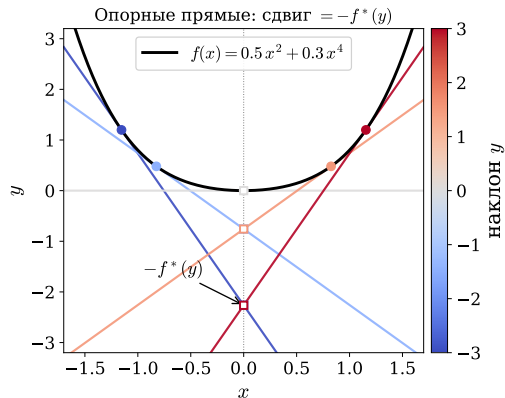


Сопряжённая к $f : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$f^*(y) = \max_x [y^T x - f(x)]$$

Figure 3: Опорная прямая наклона y , $l_y(x) = yx - f^*(y)$, касается графика снизу; её пересечение с осью ординат равно $-f^*(y)$. Цвет соответствует наклону y .

Сопряжённая функция f^* : определение и геометрия



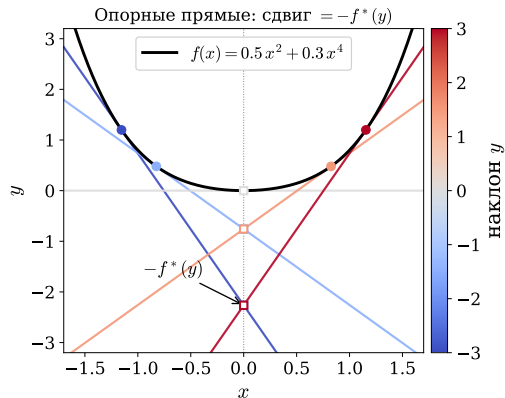
Сопряжённая к $f : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$f^*(y) = \max_x [y^T x - f(x)]$$

- При фиксированном y максимум по x достигается там, где $f'(x) = y$.

Figure 3: Опорная прямая наклона y , $l_y(x) = yx - f^*(y)$, касается графика снизу; её пересечение с осью ординат равно $-f^*(y)$. Цвет соответствует наклону y .

Сопряжённая функция f^* : определение и геометрия



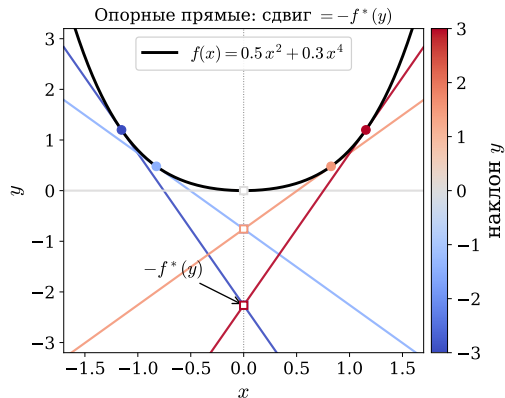
Сопряжённая к $f : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$f^*(y) = \max_x [y^T x - f(x)]$$

- При фиксированном y максимум по x достигается там, где $f'(x) = y$.
- $-f^*(y)$ есть точка пересечения опорной прямой наклона y с осью ординат.

Figure 3: Опорная прямая наклона y , $l_y(x) = yx - f^*(y)$, касается графика снизу; её пересечение с осью ординат равно $-f^*(y)$. Цвет соответствует наклону y .

Сопряжённая функция f^* : определение и геометрия



Сопряжённая к $f : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$f^*(y) = \max_x [y^T x - f(x)]$$

- При фиксированном y максимум по x достигается там, где $f'(x) = y$.
- $-f^*(y)$ есть точка пересечения опорной прямой наклона y с осью ординат.
- Если f замкнута и выпукла, то $f^{**} = f$: график f восстанавливается как верхняя огибающая своих опорных прямых.

Figure 3: Опорная прямая наклона y , $l_y(x) = yx - f^*(y)$, касается графика снизу; её пересечение с осью ординат равно $-f^*(y)$. Цвет соответствует наклону y .

Геометрическая интуиция: наклон $\leftrightarrow$ аргумент

Двойственность наклон $\leftrightarrow$ аргумент: $y_0 = f'(x_0) \Leftrightarrow x_0 = (f^*)'(y_0)$

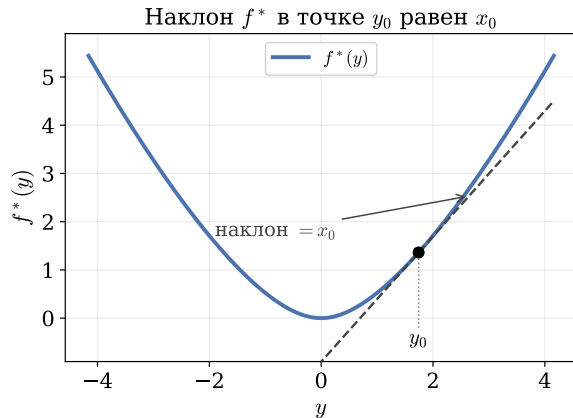
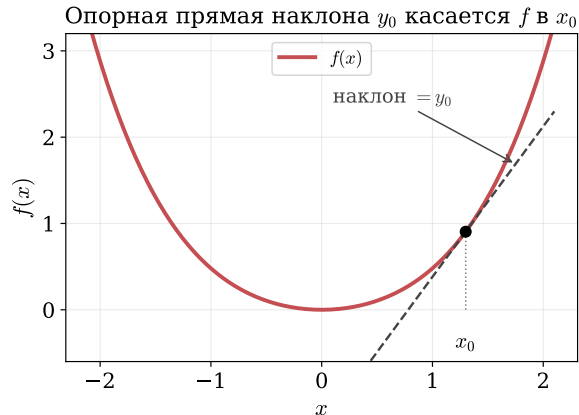
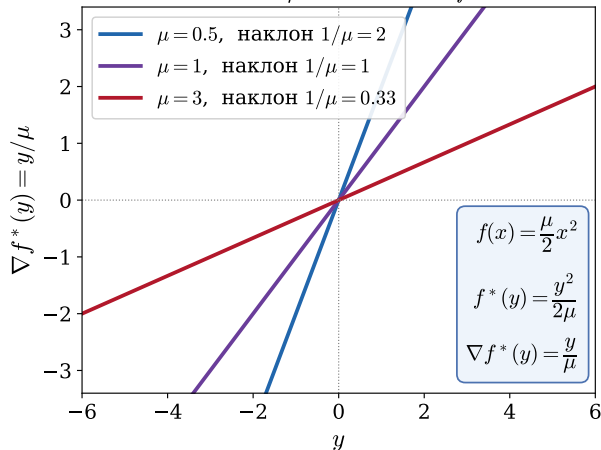


Figure 4: Для замкнутой выпуклой f : наклон f в x_0 равен $y_0 \Leftrightarrow$ наклон f^* в y_0 равен x_0 ; субдифференциалы взаимно обратны.

Сильная выпуклость f и гладкость f^*

Больше $\mu \rightarrow$ положе ∇f^*



! Important

f μ -сильно выпукла $\iff \nabla f^*$ липшицев с константой $1/\mu$.

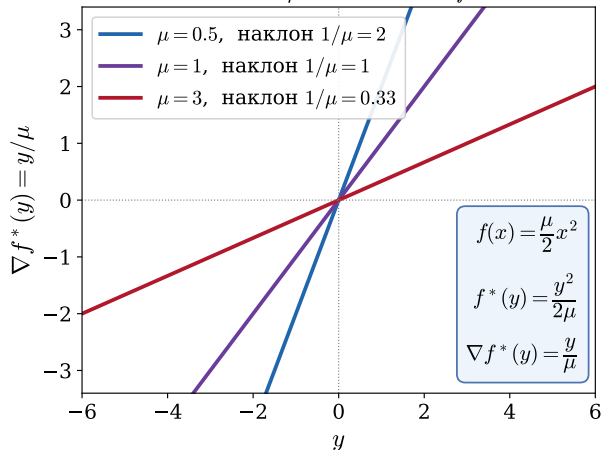
Figure 5: Пример: $f(x) = \frac{\mu}{2}x^2 \Rightarrow f^*(y) = \frac{y^2}{2\mu}$, $\nabla f^*(y) = y/\mu$.

Чем острее f (больше μ), тем положе ∇f^* (наклон $1/\mu$).

Сильная выпуклость f равносильна липшицевости ∇f^* .

Сильная выпуклость f и гладкость f^*

Больше $\mu \rightarrow$ положе ∇f^*



! Important

f μ -сильно выпукла $\iff \nabla f^*$ липшицев с константой $1/\mu$.

Почему так? $\nabla f^*(y) = \arg \min_x [f(x) - yx]$. Если f круто растёт (большое μ), малое изменение наклона y почти не сдвигает точку минимума x , поэтому ∇f^* меняется медленно (гладко).

Figure 5: Пример: $f(x) = \frac{\mu}{2}x^2 \Rightarrow f^*(y) = \frac{y^2}{2\mu}$, $\nabla f^*(y) = y/\mu$.

Чем острее f (больше μ), тем положе ∇f^* (наклон $1/\mu$).

Сильная выпуклость f равносильна липшицевости ∇f^* .

Сильная выпуклость f и гладкость f^*

Больше $\mu \rightarrow$ положе ∇f^*

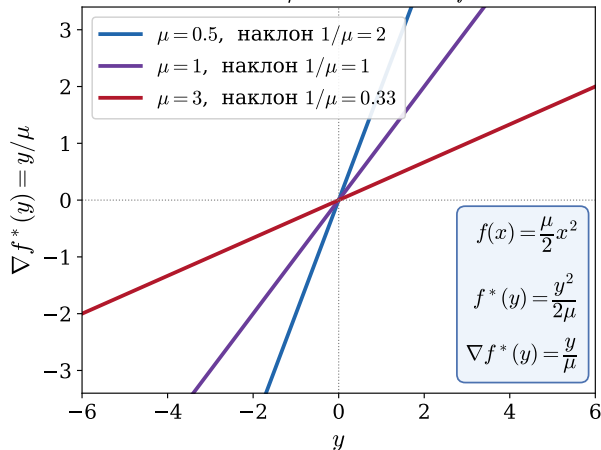


Figure 5: Пример: $f(x) = \frac{\mu}{2}x^2 \Rightarrow f^*(y) = \frac{y^2}{2\mu}$, $\nabla f^*(y) = y/\mu$.

Чем острее f (больше μ), тем положе ∇f^* (наклон $1/\mu$).

Сильная выпуклость f равносильна липшицевости ∇f^* .

! Important

f μ -сильно выпукла $\iff \nabla f^*$ липшицев с константой $1/\mu$.

Почему так? $\nabla f^*(y) = \arg \min_x [f(x) - yx]$. Если f круто растёт (большое μ), малое изменение наклона y почти не сдвигает точку минимума x , поэтому ∇f^* меняется медленно (гладко).

Зачем это в лекции: скорость двойственного подъёма есть скорость подъёма по $g(v) = -f^*(-A^T v) - b^T v$. Гладкость g следует из гладкости f^* , а та — из сильной выпуклости f . Если f не сильно выпукла, то f^* негладкая и двойственный подъём сходится сублинейно. Именно это исправляет ALM.

Двойственный градиентный подъём

Двойственный (суб)градиентный метод

Даже если двойственную задачу не удаётся вывести в замкнутой форме, можно применять к ней градиентные или субградиентные методы.

Рассмотрим задачу:

$$\min_x f(x) \quad \text{при} \quad Ax = b$$

Двойственный (суб)градиентный метод

Даже если двойственную задачу не удаётся вывести в замкнутой форме, можно применять к ней градиентные или субградиентные методы.

Рассмотрим задачу:

$$\min_x f(x) \quad \text{при} \quad Ax = b$$

Её двойственная задача:

$$\max_{\nu} -f^*(-A^T \nu) - b^T \nu$$

где f^* — сопряжённая к f . Обозначив $g(\nu) = -f^*(-A^T \nu) - b^T \nu$, заметим:

$$\partial g(\nu) = A \partial f^*(-A^T \nu) - b$$

Двойственный (суб)градиентный метод

Даже если двойственную задачу не удаётся вывести в замкнутой форме, можно применять к ней градиентные или субградиентные методы.

Рассмотрим задачу:

$$\min_x f(x) \quad \text{при} \quad Ax = b$$

Её двойственная задача:

$$\max_{\nu} -f^*(-A^T \nu) - b^T \nu$$

где f^* — сопряжённая к f . Обозначив $g(\nu) = -f^*(-A^T \nu) - b^T \nu$, заметим:

$$\partial g(\nu) = A \partial f^*(-A^T \nu) - b$$

Следовательно, используя известные свойства сопряжённых функций,

$$\partial g(\nu) = Ax - b \quad \text{где} \quad x \in \arg \min_z [f(z) + \nu^T Az]$$

Двойственный (суб)градиентный метод

Даже если двойственную задачу не удаётся вывести в замкнутой форме, можно применять к ней градиентные или субградиентные методы.

Рассмотрим задачу:

$$\min_x f(x) \quad \text{при} \quad Ax = b$$

Её двойственная задача:

$$\max_{\nu} -f^*(-A^T \nu) - b^T \nu$$

где f^* — сопряжённая к f . Обозначив $g(\nu) = -f^*(-A^T \nu) - b^T \nu$, заметим:

$$\partial g(\nu) = A \partial f^*(-A^T \nu) - b$$

Следовательно, используя известные свойства сопряжённых функций,

$$\partial g(\nu) = Ax - b \quad \text{где} \quad x \in \arg \min_z [f(z) + \nu^T Az]$$

Dual ascent:

$$x_k \in \arg \min_x [f(x) + (\nu_{k-1})^T Ax]$$

$$\nu_k = \nu_{k-1} + \alpha_k (Ax_k - b)$$

- Шаги α_k , $k = 1, 2, 3, \dots$, выбираются стандартными способами.

Двойственный (суб)градиентный метод

Даже если двойственную задачу не удаётся вывести в замкнутой форме, можно применять к ней градиентные или субградиентные методы.

Рассмотрим задачу:

$$\min_x f(x) \quad \text{при} \quad Ax = b$$

Её двойственная задача:

$$\max_{\nu} -f^*(-A^T \nu) - b^T \nu$$

где f^* — сопряжённая к f . Обозначив $g(\nu) = -f^*(-A^T \nu) - b^T \nu$, заметим:

$$\partial g(\nu) = A \partial f^*(-A^T \nu) - b$$

Следовательно, используя известные свойства сопряжённых функций,

$$\partial g(\nu) = Ax - b \quad \text{где} \quad x \in \arg \min_z [f(z) + \nu^T Az]$$

Dual ascent:

$$x_k \in \arg \min_x [f(x) + (\nu_{k-1})^T Ax]$$

$$\nu_k = \nu_{k-1} + \alpha_k (Ax_k - b)$$

- Шаги α_k , $k = 1, 2, 3, \dots$, выбираются стандартными способами.
- Проксимальные градиентные методы и ускорение (acceleration) применяются так же, как обычно.

Гессиан двойственной функции: вывод

Двойственная функция $g(\nu) = -f^*(-A^T \nu) - b^T \nu$. Найдём $\nabla^2 g(\nu)$.

Гессиан двойственной функции: вывод

Двойственная функция $g(\nu) = -f^*(-A^T \nu) - b^T \nu$. Найдём $\nabla^2 g(\nu)$.

Градиент. Так как $\frac{d}{d\nu}(-A^T \nu) = -A^T$, цепное правило даёт $\nabla_\nu f^*(-A^T \nu) = -A \nabla f^*(-A^T \nu)$, поэтому

$$\nabla g(\nu) = A \nabla f^*(-A^T \nu) - b = Ax - b, \quad x := \nabla f^*(-A^T \nu) = \arg \min_z [f(z) + \nu^T Az].$$

Гессиан двойственной функции: вывод

Двойственная функция $g(\nu) = -f^*(-A^T \nu) - b^T \nu$. Найдём $\nabla^2 g(\nu)$.

Градиент. Так как $\frac{d}{d\nu}(-A^T \nu) = -A^T$, цепное правило даёт $\nabla_\nu f^*(-A^T \nu) = -A \nabla f^*(-A^T \nu)$, поэтому

$$\nabla g(\nu) = A \nabla f^*(-A^T \nu) - b = Ax - b, \quad x := \nabla f^*(-A^T \nu) = \arg \min_z [f(z) + \nu^T Az].$$

Гессиан. Дифференцируем $\nabla g(\nu) = A \nabla f^*(-A^T \nu) - b$ ещё раз по ν (та же подстановка, якобиан $-A^T$):

$$\nabla^2 g(\nu) = A \nabla^2 f^*(-A^T \nu) (-A^T) = -A \nabla^2 f^*(-A^T \nu) A^T.$$

Гессиан двойственной функции: вывод

Двойственная функция $g(\nu) = -f^*(-A^T \nu) - b^T \nu$. Найдём $\nabla^2 g(\nu)$.

Градиент. Так как $\frac{d}{d\nu}(-A^T \nu) = -A^T$, цепное правило даёт $\nabla_{\nu} f^*(-A^T \nu) = -A \nabla f^*(-A^T \nu)$, поэтому

$$\nabla g(\nu) = A \nabla f^*(-A^T \nu) - b = Ax - b, \quad x := \nabla f^*(-A^T \nu) = \arg \min_z [f(z) + \nu^T Az].$$

Гессиан. Дифференцируем $\nabla g(\nu) = A \nabla f^*(-A^T \nu) - b$ ещё раз по ν (та же подстановка, якобиан $-A^T$):

$$\nabla^2 g(\nu) = A \nabla^2 f^*(-A^T \nu) (-A^T) = -A \nabla^2 f^*(-A^T \nu) A^T.$$

Сопряжённый гессиан обратен исходному. Для замкнутой выпуклой f с обратимым $\nabla^2 f$ имеем $x = \nabla f^*(y) \Leftrightarrow y = \nabla f(x)$. Дифференцируя тождество $y = \nabla f(\nabla f^*(y))$ по y :

$$I = \nabla^2 f(x) \nabla^2 f^*(y) \quad \Rightarrow \quad \nabla^2 f^*(y) = [\nabla^2 f(x)]^{-1}.$$

Гессиан двойственной функции: вывод

Двойственная функция $g(\nu) = -f^*(-A^T \nu) - b^T \nu$. Найдём $\nabla^2 g(\nu)$.

Градиент. Так как $\frac{d}{d\nu}(-A^T \nu) = -A^T$, цепное правило даёт $\nabla_\nu f^*(-A^T \nu) = -A \nabla f^*(-A^T \nu)$, поэтому

$$\nabla g(\nu) = A \nabla f^*(-A^T \nu) - b = Ax - b, \quad x := \nabla f^*(-A^T \nu) = \arg \min_z [f(z) + \nu^T Az].$$

Гессиан. Дифференцируем $\nabla g(\nu) = A \nabla f^*(-A^T \nu) - b$ ещё раз по ν (та же подстановка, якобиан $-A^T$):

$$\nabla^2 g(\nu) = A \nabla^2 f^*(-A^T \nu) (-A^T) = -A \nabla^2 f^*(-A^T \nu) A^T.$$

Сопряжённый гессиан обратен исходному. Для замкнутой выпуклой f с обратимым $\nabla^2 f$ имеем $x = \nabla f^*(y) \Leftrightarrow y = \nabla f(x)$. Дифференцируя тождество $y = \nabla f(\nabla f^*(y))$ по y :

$$I = \nabla^2 f(x) \nabla^2 f^*(y) \quad \Rightarrow \quad \nabla^2 f^*(y) = [\nabla^2 f(x)]^{-1}.$$

Итог. Подставляя $y = -A^T \nu$ и $x = \nabla f^*(-A^T \nu)$:

$$\nabla^2 g(\nu) = -A [\nabla^2 f(x)]^{-1} A^T$$

Для квадратичной f гессиан $\nabla^2 f$ постоянен, и $\nabla^2 g(\nu) = -A(\nabla^2 f)^{-1} A^T$. (Матрица $\preceq 0$ — двойственную мы максимизируем.)

Какая обусловленность управляет скоростью?

Двойственный подъём есть градиентный подъём по $g(\nu)$. Его скорость определяется обусловленностью g , а не f .

Какая обусловленность управляет скоростью?

Двойственный подъём есть градиентный подъём по $g(\nu)$. Его скорость определяется обусловленностью g , а не f .

Для квадратичной f с гессианом $\nabla^2 f$ (собственные значения в $[\mu, L]$) при ограничении $Ax = b$ гессиан двойственной функции равен

$$\nabla^2 g(\nu) = -A (\nabla^2 f)^{-1} A^T.$$

- Собственные значения лежат в $\left[\frac{\sigma_{\min}^2(A)}{L}, \frac{\sigma_{\max}^2(A)}{\mu} \right]$.

Какая обусловленность управляет скоростью?

Двойственный подъём есть градиентный подъём по $g(\nu)$. Его скорость определяется обусловленностью g , а не f .

Для квадратичной f с гессианом $\nabla^2 f$ (собственные значения в $[\mu, L]$) при ограничении $Ax = b$ гессиан двойственной функции равен

$$\nabla^2 g(\nu) = -A (\nabla^2 f)^{-1} A^T.$$

- Собственные значения лежат в $\left[\frac{\sigma_{\min}^2(A)}{L}, \frac{\sigma_{\max}^2(A)}{\mu} \right]$.
- **Обусловленность двойственной задачи:**

$$\kappa_d = \frac{\sigma_{\max}^2(A)}{\sigma_{\min}^2(A)} \cdot \frac{L}{\mu} = \kappa(A)^2 \cdot \kappa(\nabla^2 f).$$

Какая обусловленность управляет скоростью?

Двойственный подъём есть градиентный подъём по $g(\nu)$. Его скорость определяется обусловленностью g , а не f .

Для квадратичной f с гессианом $\nabla^2 f$ (собственные значения в $[\mu, L]$) при ограничении $Ax = b$ гессиан двойственной функции равен

$$\nabla^2 g(\nu) = -A (\nabla^2 f)^{-1} A^T.$$

- Собственные значения лежат в $\left[\frac{\sigma_{\min}^2(A)}{L}, \frac{\sigma_{\max}^2(A)}{\mu} \right]$.
- **Обусловленность двойственной задачи:**

$$\kappa_d = \frac{\sigma_{\max}^2(A)}{\sigma_{\min}^2(A)} \cdot \frac{L}{\mu} = \kappa(A)^2 \cdot \kappa(\nabla^2 f).$$

Какая обусловленность управляет скоростью?

Двойственный подъём есть градиентный подъём по $g(\nu)$. Его скорость определяется обусловленностью g , а не f .

Для квадратичной f с гессианом $\nabla^2 f$ (собственные значения в $[\mu, L]$) при ограничении $Ax = b$ гессиан двойственной функции равен

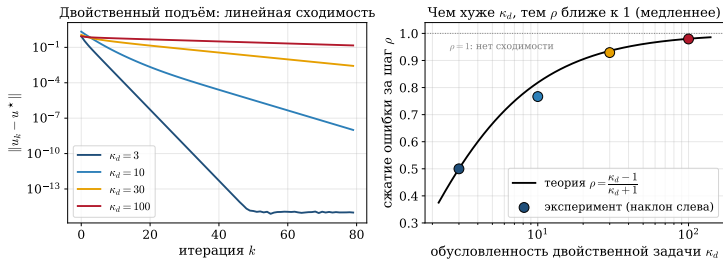
$$\nabla^2 g(\nu) = -A (\nabla^2 f)^{-1} A^T.$$

- Собственные значения лежат в $\left[\frac{\sigma_{\min}^2(A)}{L}, \frac{\sigma_{\max}^2(A)}{\mu} \right]$.
- **Обусловленность двойственной задачи:**

$$\kappa_d = \frac{\sigma_{\max}^2(A)}{\sigma_{\min}^2(A)} \cdot \frac{L}{\mu} = \kappa(A)^2 \cdot \kappa(\nabla^2 f).$$

Вывод. Скорость двойственного подъёма портят **два** источника: плохая обусловленность самой f ($\kappa(\nabla^2 f) = L/\mu$) и плохая обусловленность матрицы ограничений A (вклад $\kappa(A)^2$). Даже при идеально обусловленной f ($\kappa(\nabla^2 f) = 1$) плохая A даёт медленный двойственный подъём.

Скорости двойственного градиентного подъёма: теория и эмпирика



Две группы кривых отвечают двум гарантиям:

- f **сильно выпукла** $\Rightarrow O(1/\epsilon)$ (при малом κ_d быстро).

Figure 6: Слева: сходимость двойственного подъёма для квадратичных задач с разной обусловленностью κ_d — чем больше κ_d , тем положе прямая (медленнее). Справа: коэффициент сжатия ошибки за шаг ρ как функция κ_d . Чёрная кривая — теория $\rho = (\kappa_d - 1)/(\kappa_d + 1)$, точки — измеренный наклон лог-графика слева. С ростом κ_d имеем $\rho \rightarrow 1$ (сходимости почти нет); теория — оценка сверху.

Скорости двойственного градиентного подъёма: теория и эмпирика

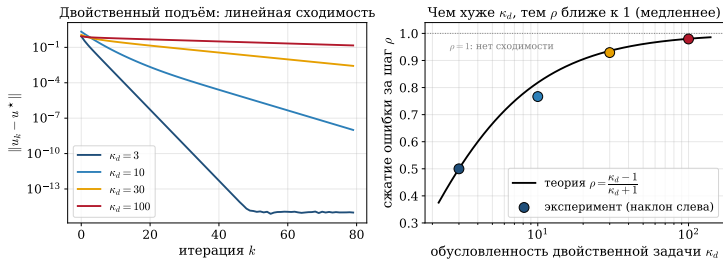
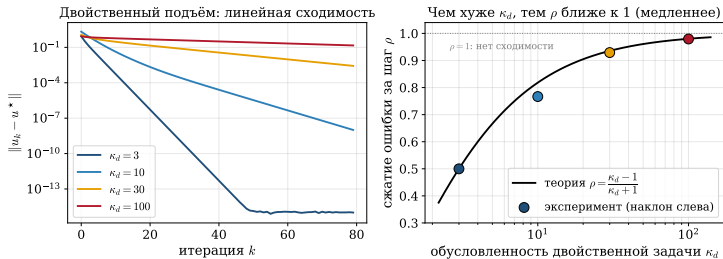


Figure 6: Слева: сходимость двойственного подъёма для квадратичных задач с разной обусловленностью κ_d — чем больше κ_d , тем положение прямая (медленнее). Справа: коэффициент сжатия ошибки за шаг ρ как функция κ_d . Чёрная кривая — теория $\rho = (\kappa_d - 1)/(\kappa_d + 1)$, точки — измеренный наклон лог-графика слева. С ростом κ_d имеем $\rho \rightarrow 1$ (сходимости почти нет); теория — оценка сверху.

Две группы кривых отвечают двум гарантиям:

- f сильно выпукла $\Rightarrow O(1/\epsilon)$ (при малом κ_d быстро).
- f сильно выпукла и ∇f липшицев $\Rightarrow$ линейно, $O(\log(1/\epsilon))$.

Скорости двойственного градиентного подъёма: теория и эмпирика



Две группы кривых отвечают двум гарантиям:

- f сильно выпукла $\Rightarrow O(1/\epsilon)$ (при малом κ_d быстро).
- f сильно выпукла и ∇f липшицев $\Rightarrow$ линейно, $O(\log(1/\epsilon))$.

Figure 6: Слева: сходимость двойственного подъёма для квадратичных задач с разной обусловленностью κ_d — чем больше κ_d , тем положение прямая (медленнее). Справа: коэффициент сжатия ошибки за шаг ρ как функция κ_d . Чёрная кривая — теория $\rho = (\kappa_d - 1)/(\kappa_d + 1)$, точки — измеренный наклон лог-графика слева. С ростом κ_d имеем $\rho \rightarrow 1$ (сходимости почти нет); теория — оценка сверху.

Скорости двойственного градиентного подъёма: теория и эмпирика

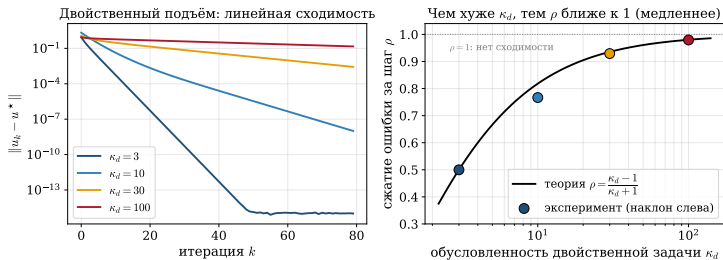


Figure 6: Слева: сходимость двойственного подъёма для квадратичных задач с разной обусловленностью κ_d — чем больше κ_d , тем положение прямая (медленнее). Справа: коэффициент сжатия ошибки за шаг ρ как функция κ_d . Чёрная кривая — теория $\rho = (\kappa_d - 1)/(\kappa_d + 1)$, точки — измеренный наклон лог-графика слева. С ростом κ_d имеем $\rho \rightarrow 1$ (сходимости почти нет); теория — оценка сверху.

Две группы кривых отвечают двум гарантиям:

- f сильно выпукла $\Rightarrow O(1/\epsilon)$ (при малом κ_d быстро).
- f сильно выпукла и ∇f липшицев $\Rightarrow$ линейно, $O(\log(1/\epsilon))$.

При большом κ_d кривая почти плоская. Сходимость указана в двойственной задаче; в прямой она требует дополнительных условий.

Чувствительность к шагу: двойственный градиентный подъём и ALM

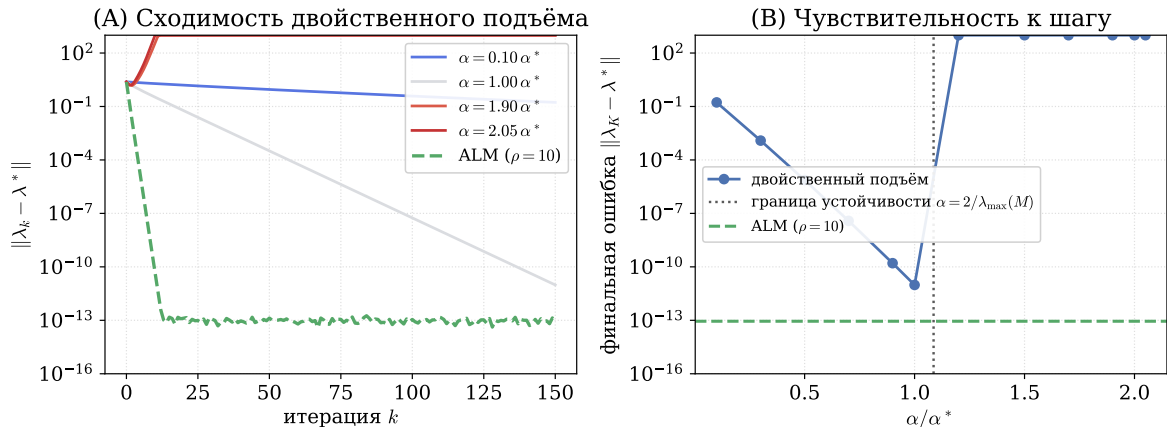


Figure 7: Окно допустимых шагов α узкое, граница устойчивости резкая ($\alpha \approx 2/L_g$). Слишком большой шаг даёт расходимость, слишком малый — медленную сходимость. У ALM выбор $\alpha = \rho$ устойчив по построению.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.
- **Размерность.** В SVM двойственная задача меньше и допускает переход к ядрам; решение разрежено.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.
- **Размерность.** В SVM двойственная задача меньше и допускает переход к ядрам; решение разрежено.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.
- **Размерность.** В SVM двойственная задача меньше и допускает переход к ядрам; решение разрежено.

Цена за это:

- Нужна **сильная выпуклость** f для гладкости g и постоянного шага.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.
- **Размерность.** В SVM двойственная задача меньше и допускает переход к ядрам; решение разрежено.

Цена за это:

- Нужна **сильная выпуклость** f для гладкости g и постоянного шага.
- Сходимость гарантируется в **двойственной** задаче; восстановление x^* требует дополнительных условий.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.
- **Размерность.** В SVM двойственная задача меньше и допускает переход к ядрам; решение разрежено.

Цена за это:

- Нужна **сильная выпуклость** f для гладкости g и постоянного шага.
- Сходимость гарантируется в **двойственной** задаче; восстановление x^* требует дополнительных условий.
- Плохо обусловленная A убивает скорость двойственного шага.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.
- **Размерность.** В SVM двойственная задача меньше и допускает переход к ядрам; решение разрежено.

Цена за это:

- Нужна **сильная выпуклость** f для гладкости g и постоянного шага.
- Сходимость гарантируется в **двойственной** задаче; восстановление x^* требует дополнительных условий.
- Плохо обусловленная A убивает скорость двойственного шага.

Зачем двойственный градиентный подъём, если есть прямые методы?

Где двойственный подъём выигрывает:

- **Декомпозиция.** Если $f(x) = \sum_i f_i(x_i)$, то x -шаг распадается на B независимых подзадач.
- **Снятие связности.** Сложное ограничение $Ax = b$ заменяется ценой ν , после чего минимизация f становится простой.
- **Размерность.** В SVM двойственная задача меньше и допускает переход к ядрам; решение разрежено.

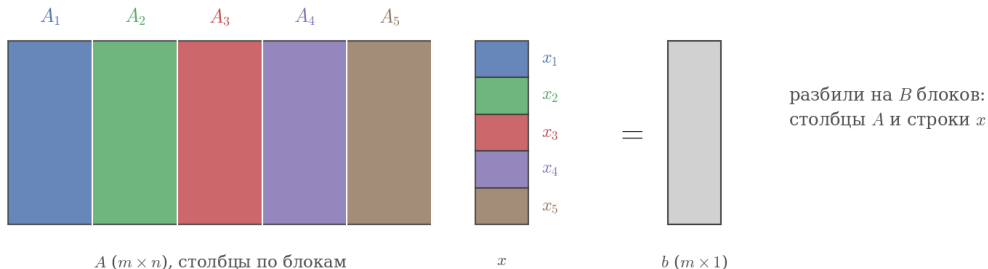
Цена за это:

- Нужна **сильная выпуклость** f для гладкости g и постоянного шага.
- Сходимость гарантируется в **двойственной** задаче; восстановление x^* требует дополнительных условий.
- Плохо обусловленная A убивает скорость двойственного шага.

Поэтому дальше: ALM добавляет штраф $\frac{\rho}{2} \|Ax - b\|^2$ и снимает требование сильной выпуклости, а ADMM возвращает декомпозицию.

Двойственная декомпозиция

Идея: развязать связь ценой



A ($m \times n$), столбцы по блокам

x

b ($m \times 1$)

$$Ax = A_1x_1 + A_2x_2 + A_3x_3 + A_4x_4 + A_5x_5 = b$$

целевая функция разделима: $f(x) = \sum_{i=1}^B f_i(x_i)$ — каждый x_i только в своём f_i

единственная связь блоков — общий ресурс $\sum_{i=1}^B A_i x_i = b$

Зачем декомпозиция? Завод и пять цехов

Монолит. Один центр знает всё: целевые функции всех агентов $f_1, \dots, f_B$, все переменные, все ограничения. Решает гигантскую задачу целиком.

Зачем декомпозиция? Завод и пять цехов

Монолит. Один центр знает всё: целевые функции всех агентов $f_1, \dots, f_B$, все переменные, все ограничения. Решает гигантскую задачу целиком.

$$\min_{x_1, \dots, x_B} \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i = b$$

Зачем декомпозиция? Завод и пять цехов

Монолит. Один центр знает всё: целевые функции всех агентов $f_1, \dots, f_B$, все переменные, все ограничения. Решает гигантскую задачу целиком.

$$\min_{x_1, \dots, x_B} \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i = b$$

Проблема. f_i — закрытая внутренняя информация цеха (своя технология, свои издержки). Задача огромна и неразделима из-за общего ресурса b .

Зачем декомпозиция? Завод и пять цехов

Монолит. Один центр знает всё: целевые функции всех агентов $f_1, \dots, f_B$, все переменные, все ограничения. Решает гигантскую задачу целиком.

$$\min_{x_1, \dots, x_B} \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i = b$$

Проблема. f_i — закрытая внутренняя информация цеха (своя технология, свои издержки). Задача огромна и неразделима из-за общего ресурса b .

Идея двойственности. Ограничение $\sum_i A_i x_i = b$ связывает агентов. Без него каждый цех решал бы свою маленькую задачу сам.

Зачем декомпозиция? Завод и пять цехов

Монолит. Один центр знает всё: целевые функции всех агентов $f_1, \dots, f_B$, все переменные, все ограничения. Решает гигантскую задачу целиком.

$$\min_{x_1, \dots, x_B} \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i = b$$

Проблема. f_i — закрытая внутренняя информация цеха (своя технология, свои издержки). Задача огромна и неразделима из-за общего ресурса b .

Идея двойственности. Ограничение $\sum_i A_i x_i = b$ связывает агентов. Без него каждый цех решал бы свою маленькую задачу сам.

Развяжем ценой λ . Центр объявляет *цену ресурса* λ . Каждый цех платит $\lambda^T A_i x_i$ и решает локально:

$$x_i = \arg \min_{x_i} [f_i(x_i) + \lambda^T A_i x_i]$$

Зачем декомпозиция? Завод и пять цехов

Монолит. Один центр знает всё: целевые функции всех агентов $f_1, \dots, f_B$, все переменные, все ограничения. Решает гигантскую задачу целиком.

$$\min_{x_1, \dots, x_B} \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i = b$$

Проблема. f_i — закрытая внутренняя информация цеха (своя технология, свои издержки). Задача огромна и неразделима из-за общего ресурса b .

Идея двойственности. Ограничение $\sum_i A_i x_i = b$ связывает агентов. Без него каждый цех решал бы свою маленькую задачу сам.

Развяжем ценой λ . Центр объявляет *цену ресурса* λ . Каждый цех платит $\lambda^T A_i x_i$ и решает локально:

$$x_i = \arg \min_{x_i} [f_i(x_i) + \lambda^T A_i x_i]$$

Центр видит только суммарный спрос $\sum_i A_i x_i$, а не f_i . Конфиденциальность сохранена, задача распалась на B независимых подзадач.

Двойственная декомпозиция

Сепарабельная задача с общим ограничением:

$$\min_x \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad Ax = b$$

Двойственная декомпозиция

Сепарабельная задача с общим ограничением:

$$\min_x \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad Ax = b$$

Здесь $x = (x_1, \dots, x_B) \in \mathbb{R}^n$ разбивается на B блоков переменных, причём $x_i \in \mathbb{R}^{n_i}$ (двойственная переменная ν — та же цена ресурса λ из экономической интерпретации). Матрицу A также можно разбить согласованным образом:

$$A = [A_1 \dots A_B], \text{ где } A_i \in \mathbb{R}^{m \times n_i}$$

Двойственная декомпозиция

Сепарабельная задача с общим ограничением:

$$\min_x \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad Ax = b$$

Здесь $x = (x_1, \dots, x_B) \in \mathbb{R}^n$ разбивается на B блоков переменных, причём $x_i \in \mathbb{R}^{n_i}$ (двойственная переменная ν — та же цена ресурса λ из экономической интерпретации). Матрицу A также можно разбить согласованным образом:

$$A = [A_1 \dots A_B], \text{ где } A_i \in \mathbb{R}^{m \times n_i}$$

Минимизация раскладывается на B отдельных задач:

$$x^{\text{new}} \in \arg \min_x \left(\sum_{i=1}^B f_i(x_i) + \nu^T Ax \right)$$

$$\Rightarrow x_i^{\text{new}} \in \arg \min_{x_i} (f_i(x_i) + \nu^T A_i x_i), \quad i = 1, \dots, B$$

$$x_i^k \in \arg \min_{x_i} (f_i(x_i) + (\nu^{k-1})^T A_i x_i), \quad i = 1, \dots, B$$

$$\nu^k = \nu^{k-1} + \alpha_k \left(\sum_{i=1}^B A_i x_i^k - b \right)$$

Двойственная декомпозиция

Сепарабельная задача с общим ограничением:

$$\min_x \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad Ax = b$$

Здесь $x = (x_1, \dots, x_B) \in \mathbb{R}^n$ разбивается на B блоков переменных, причём $x_i \in \mathbb{R}^{n_i}$ (двойственная переменная ν — та же цена ресурса λ из экономической интерпретации). Матрицу A также можно разбить согласованным образом:

$$A = [A_1 \dots A_B], \text{ где } A_i \in \mathbb{R}^{m \times n_i}$$

Минимизация раскладывается на B отдельных задач:

$$x^{\text{new}} \in \arg \min_x \left(\sum_{i=1}^B f_i(x_i) + \nu^T Ax \right)$$

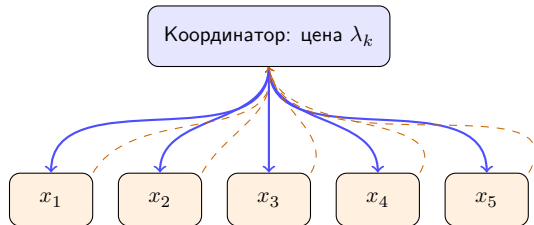
$$\Rightarrow x_i^{\text{new}} \in \arg \min_{x_i} (f_i(x_i) + \nu^T A_i x_i), \quad i = 1, \dots, B$$

$$x_i^k \in \arg \min_{x_i} (f_i(x_i) + (\nu^{k-1})^T A_i x_i), \quad i = 1, \dots, B$$

$$\nu^k = \nu^{k-1} + \alpha_k \left(\sum_{i=1}^B A_i x_i^k - b \right)$$

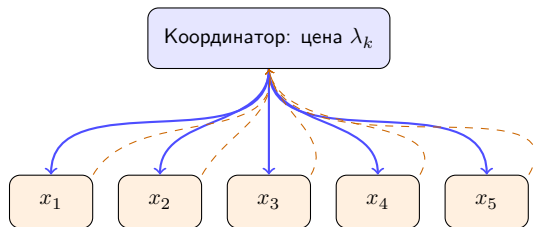
Минимизация $\sum_i f_i$ при общем ν распадается на B независимых подзадач: каждый блок x_i решается отдельно. Один раунд состоит из рассылки цены ν , локальных шагов и сбора вкладов $A_i x_i$ (см. схему далее).

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор



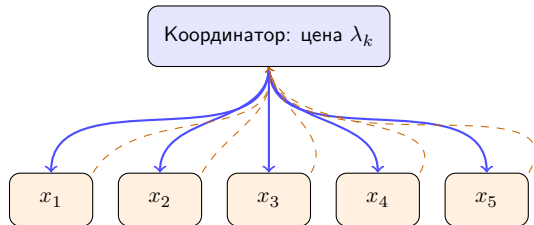
- **Рассылка** (синие): разослать цену λ_k всем.

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор



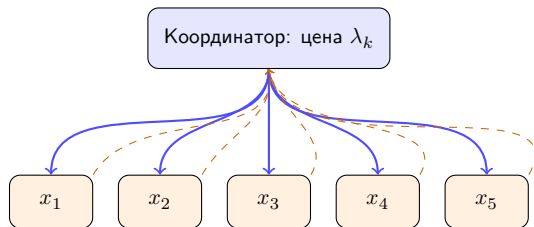
- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно:** каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор



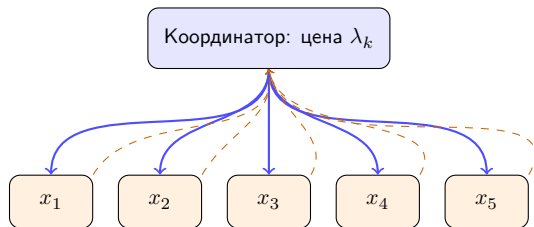
- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно**: каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.
- **Сбор** (пунктир): собрать вклады $A_i x_i^k$ и обновить цену.

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор



- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно**: каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.
- **Сбор** (пунктир): собрать вклады $A_i x_i^k$ и обновить цену.

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор

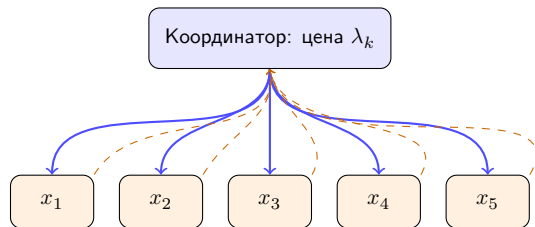


- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно:** каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.
- **Сбор** (пунктир): собрать вклады $A_i x_i^k$ и обновить цену.

Что хорошо:

- Стоимость раунда = $O(\text{макс. время агента})$, а не суммы их времён, что естественно для параллельных схем (распределённые кластеры, GPU).

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор

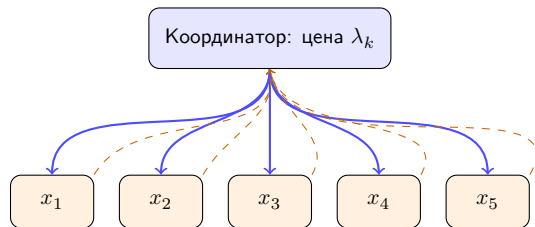


- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно**: каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.
- **Сбор** (пунктир): собрать вклады $A_i x_i^k$ и обновить цену.

Что хорошо:

- Стоимость раунда = $O(\text{макс. время агента})$, а не суммы их времён, что естественно для параллельных схем (распределённые кластеры, GPU).
- Агенты не раскрывают f_i , а сообщают только $A_i x_i$. Так сохраняется **конфиденциальность**.

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор

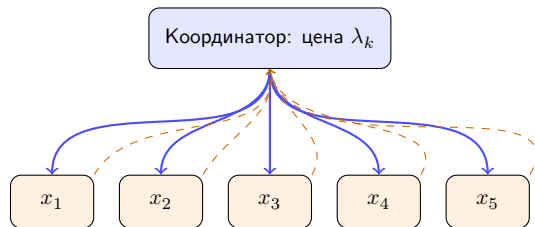


- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно**: каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.
- **Сбор** (пунктир): собрать вклады $A_i x_i^k$ и обновить цену.

Что хорошо:

- Стоимость раунда = $O(\text{макс. время агента})$, а не суммы их времён, что естественно для параллельных схем (распределённые кластеры, GPU).
- Агенты не раскрывают f_i , а сообщают только $A_i x_i$. Так сохраняется **конфиденциальность**.

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор



- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно**: каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.
- **Сбор** (пунктир): собрать вклады $A_i x_i^k$ и обновить цену.

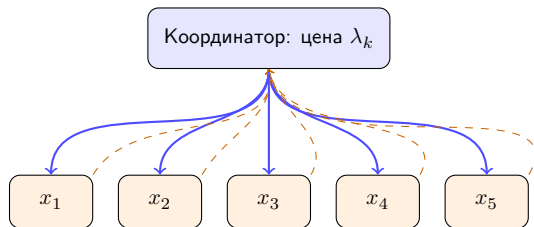
Что хорошо:

- Стоимость раунда = $O(\text{макс. время агента})$, а не суммы их времён, что естественно для параллельных схем (распределённые кластеры, GPU).
- Агенты не раскрывают f_i , а сообщают только $A_i x_i$. Так сохраняется **конфиденциальность**.

Что плохо:

- Координатор есть узкое место (синхронный барьер каждый раунд).

Один раунд: рассылка $\rightarrow$ локальный шаг $\rightarrow$ сбор



- **Рассылка** (синие): разослать цену λ_k всем.
- **Локально, параллельно**: каждый агент решает $x_i^k \in \arg \min [f_i(x_i) + \lambda_k^T A_i x_i]$.
- **Сбор** (пунктир): собрать вклады $A_i x_i^k$ и обновить цену.

Что хорошо:

- Стоимость раунда = $O(\text{макс. время агента})$, а не суммы их времён, что естественно для параллельных схем (распределённые кластеры, GPU).
- Агенты не раскрывают f_i , а сообщают только $A_i x_i$. Так сохраняется **конфиденциальность**.

Что плохо:

- Координатор есть узкое место (синхронный барьер каждый раунд).
- Скорость совпадает со скоростью двойственного подъёма: без сильной выпуклости f_i сходимость сублинейная.

Двойственная декомпозиция в действии

Координация цен: $B=5$ агентов делят ресурс $\sum_i x_i = 5$

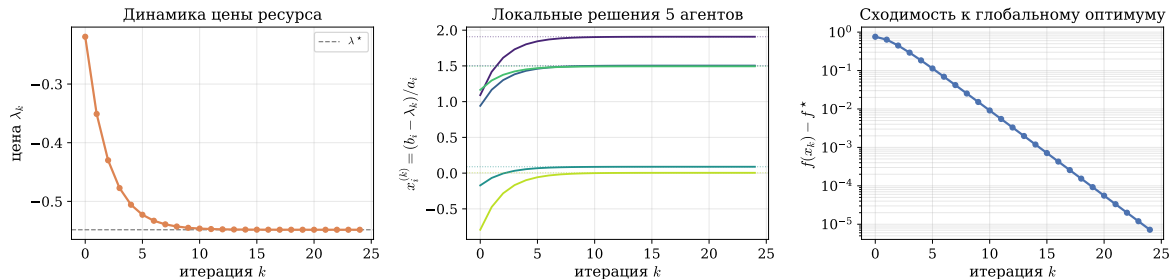


Figure 9: $B = 5$ агентов с локальными квадратичными целевыми функциями $f_i(x_i) = \frac{1}{2}a_i x_i^2 - b_i x_i$ делят общий ресурс $\sum_i x_i = 5$. Слева: динамика цены (аукционист); в центре: реакция агентов в замкнутой форме $x_i^{(k)} = (b_i - \lambda_k)/a_i$; справа: линейная сходимость к равновесию.

Рыночный механизм: нащупывание цены (tâtonnement)

Аукционист не знает целевых функций f_i : он видит лишь суммарный спрос и управляет **одной ценой** $\lambda \geq 0$.
Один такт нащупывания:

Рыночный механизм: нащупывание цены (tâtonnement)

Аукционист не знает целевых функций f_i : он видит лишь суммарный спрос и управляет **одной ценой** $\lambda \geq 0$.
Один такт нащупывания:

1. Объявляет цену λ_k .

Рыночный механизм: нащупывание цены (tâtonnement)

Аукционист не знает целевых функций f_i : он видит лишь суммарный спрос и управляет **одной ценой** $\lambda \geq 0$.
Один такт нащупывания:

1. Объявляет цену λ_k .
2. Каждый агент даёт лучший ответ $x_i(\lambda_k) = \arg \min_{x_i} [f_i(x_i) + \lambda_k^\top A_i x_i]$.

Рыночный механизм: нащупывание цены (tâtonnement)

Аукционист не знает целевых функций f_i : он видит лишь суммарный спрос и управляет **одной ценой** $\lambda \geq 0$.
Один такт нащупывания:

1. Объявляет цену λ_k .
2. Каждый агент даёт лучший ответ $x_i(\lambda_k) = \arg \min_{x_i} [f_i(x_i) + \lambda_k^\top A_i x_i]$.
3. Аукционист измеряет дисбаланс «спрос минус предложение» $r_k = \sum_i A_i x_i(\lambda_k) - b$.

Рыночный механизм: нащупывание цены (tâtonnement)

Аукционист не знает целевых функций f_i : он видит лишь суммарный спрос и управляет **одной ценой** $\lambda \geq 0$.
Один такт нащупывания:

1. Объявляет цену λ_k .
2. Каждый агент даёт лучший ответ $x_i(\lambda_k) = \arg \min_{x_i} [f_i(x_i) + \lambda_k^\top A_i x_i]$.
3. Аукционист измеряет дисбаланс «спрос минус предложение» $r_k = \sum_i A_i x_i(\lambda_k) - b$.
4. Корректирует цену: при перегрузе ($r_k > 0$) цена идёт вверх, при недогрузе — вниз:

$$\lambda_{k+1} = (\lambda_k + t r_k)_+.$$

Рыночный механизм: нащупывание цены (tâtonnement)

Аукционист не знает целевых функций f_i : он видит лишь суммарный спрос и управляет **одной ценой** $\lambda \geq 0$.
Один такт нащупывания:

1. Объявляет цену λ_k .
2. Каждый агент даёт лучший ответ $x_i(\lambda_k) = \arg \min_{x_i} [f_i(x_i) + \lambda_k^\top A_i x_i]$.
3. Аукционист измеряет дисбаланс «спрос минус предложение» $r_k = \sum_i A_i x_i(\lambda_k) - b$.
4. Корректирует цену: при перегрузе ($r_k > 0$) цена идёт вверх, при недогрузе — вниз:

$$\lambda_{k+1} = (\lambda_k + t r_k)_+.$$

Рыночный механизм: нащупывание цены (tâtonnement)

Аукционист не знает целевых функций f_i : он видит лишь суммарный спрос и управляет **одной ценой** $\lambda \geq 0$.
Один такт нащупывания:

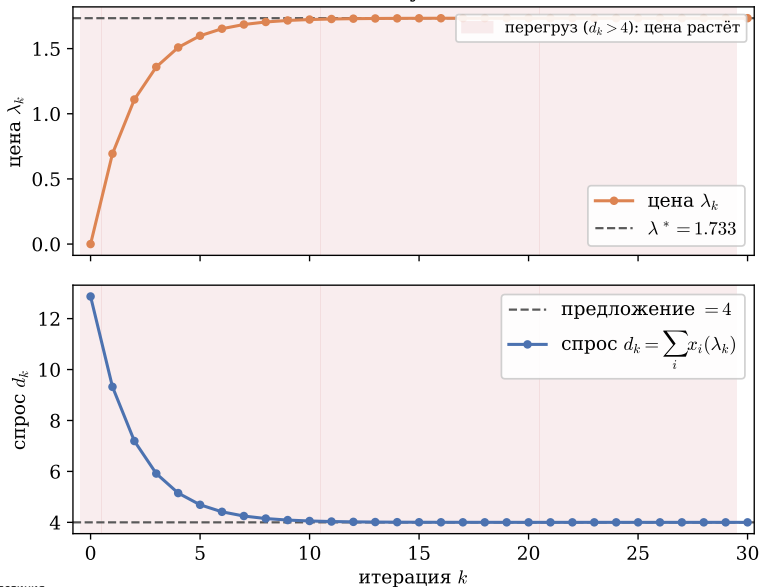
1. Объявляет цену λ_k .
2. Каждый агент даёт лучший ответ $x_i(\lambda_k) = \arg \min_{x_i} [f_i(x_i) + \lambda_k^\top A_i x_i]$.
3. Аукционист измеряет дисбаланс «спрос минус предложение» $r_k = \sum_i A_i x_i(\lambda_k) - b$.
4. Корректирует цену: при перегрузе ($r_k > 0$) цена идёт вверх, при недогрузе — вниз:

$$\lambda_{k+1} = (\lambda_k + t r_k)_+.$$

Это **двойственный подъём** для $\min \sum_i f_i(x_i)$ при $\sum_i A_i x_i = b$: цена есть двойственная переменная, а рыночное равновесие ($r_k = 0$) есть оптимум. Отрицательная цена бессмысленна, отсюда $\lambda \geq 0$.

Рыночный механизм в действии

Рыночное нащупывание цены



Ограничения-неравенства

Задача оптимизации:

$$\min_x \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i \leq b$$

Ограничения-неравенства

Задача оптимизации:

$$\min_x \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i \leq b$$

В двойственной декомпозиции (проецируемый субградиентный метод) итерации записываются так:

- Шаг обновления прямой переменной:

$$x_i^k \in \arg \min_{x_i} \left[f_i(x_i) + (\nu^{k-1})^T A_i x_i \right], \quad i = 1, \dots, B$$

Ограничения-неравенства

Задача оптимизации:

$$\min_x \sum_{i=1}^B f_i(x_i) \quad \text{при} \quad \sum_{i=1}^B A_i x_i \leq b$$

В двойственной декомпозиции (проецируемый субградиентный метод) итерации записываются так:

- Шаг обновления прямой переменной:

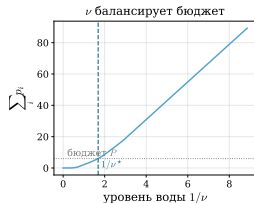
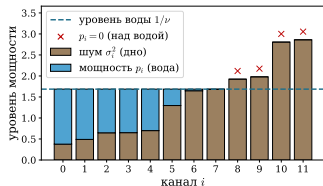
$$x_i^k \in \arg \min_{x_i} \left[f_i(x_i) + (\nu^{k-1})^T A_i x_i \right], \quad i = 1, \dots, B$$

- Шаг обновления двойственной переменной:

$$\nu^k = \left(\nu^{k-1} + \alpha_k \left(\sum_{i=1}^B A_i x_i^k - b \right) \right)_+$$

где $(\nu)_+$ — положительная часть. Проекция удерживает $\lambda \geq 0$: отрицательная цена ресурса бессмысленна.

Дополняющая нежесткость: активные и неактивные ограничения

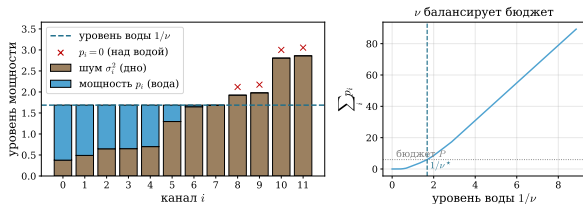


Дополняющая нежесткость: в оптимуме

$$\lambda_j^* s_j^* = 0.$$

Figure 11: Ограничение $p_i \geq 0$: у каналов с $\times$ (над водой) оно **активно** ($p_i = 0$, множитель $\lambda_i > 0$); у каналов под водой $\lambda_i = 0$, но $p_i > 0$ (запас). Произведение $\lambda_i p_i = 0$. Уровень воды задаётся отдельным множителем бюджета ν .

Дополняющая нежёсткость: активные и неактивные ограничения



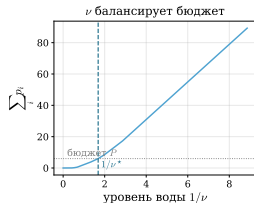
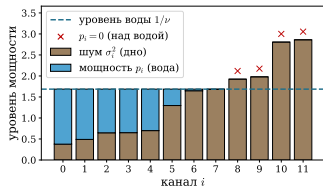
Дополняющая нежёсткость: в оптимуме

$$\lambda_j^* s_j^* = 0.$$

- $\lambda_j^* > 0 \Rightarrow s_j^* = 0$: ресурс **дефицитен**, ограничение **активно**.

Figure 11: Ограничение $p_i \geq 0$: у каналов с $\times$ (над водой) оно **активно** ($p_i = 0$, множитель $\lambda_i > 0$); у каналов под водой $\lambda_i = 0$, но $p_i > 0$ (запас). Произведение $\lambda_i p_i = 0$. Уровень воды задаётся отдельным множителем бюджета ν .

Дополняющая нежесткость: активные и неактивные ограничения



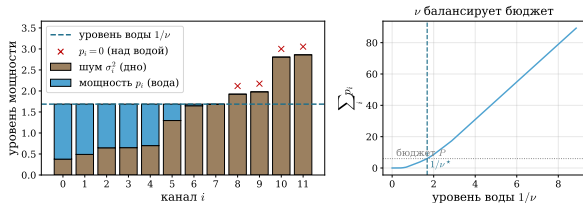
Дополняющая нежесткость: в оптимуме

$$\lambda_j^* s_j^* = 0.$$

- $\lambda_j^* > 0 \Rightarrow s_j^* = 0$: ресурс **дефицитен**, ограничение **активно**.
- $\lambda_j^* = 0 \Rightarrow s_j^* \geq 0$: ресурс **в избытке**, ограничение **неактивно**.

Figure 11: Ограничение $p_i \geq 0$: у каналов с $\times$ (над водой) оно **активно** ($p_i = 0$, множитель $\lambda_i > 0$); у каналов под водой $\lambda_i = 0$, но $p_i > 0$ (запас). Произведение $\lambda_i p_i = 0$. Уровень воды задаётся отдельным множителем бюджета ν .

Дополняющая нежесткость: активные и неактивные ограничения



Дополняющая нежесткость: в оптимуме

$$\lambda_j^* s_j^* = 0.$$

- $\lambda_j^* > 0 \Rightarrow s_j^* = 0$: ресурс **дефицитен**, ограничение **активно**.
- $\lambda_j^* = 0 \Rightarrow s_j^* \geq 0$: ресурс **в избытке**, ограничение **неактивно**.

Figure 11: Ограничение $p_i \geq 0$: у каналов с $\times$ (над водой) оно **активно** ($p_i = 0$, множитель $\lambda_i > 0$); у каналов под водой $\lambda_i = 0$, но $p_i > 0$ (запас). Произведение $\lambda_i p_i = 0$. Уровень воды задаётся отдельным множителем бюджета ν .

Дополняющая нежесткость: активные и неактивные ограничения

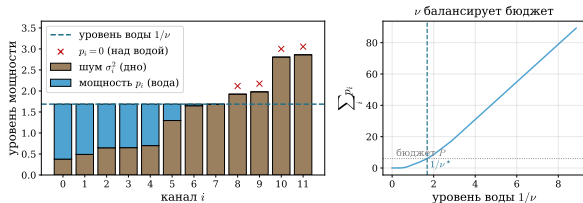


Figure 11: Ограничение $p_i \geq 0$: у каналов с $\times$ (над водой) оно **активно** ($p_i = 0$, множитель $\lambda_i > 0$); у каналов под водой $\lambda_i = 0$, но $p_i > 0$ (запас). Произведение $\lambda_i p_i = 0$. Уровень воды задаётся отдельным множителем бюджета ν .

Дополняющая нежесткость: в оптимуме

$$\lambda_j^* s_j^* = 0.$$

- $\lambda_j^* > 0 \Rightarrow s_j^* = 0$: ресурс **дефицитен**, ограничение **активно**.
- $\lambda_j^* = 0 \Rightarrow s_j^* \geq 0$: ресурс **в избытке**, ограничение **неактивно**.

Произведение «цена $\times$ запас» равно нулю: либо ограничение активно (цена > 0), либо есть запас (цена $= 0$).

Метод модифицированной функции Лагранжа (augmented Lagrangian method, ALM)

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен,
только если лагранжиан имеет минимум по x .

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен,
только если лагранжиан имеет минимум по x .

- Если f **просто выпукла** (не сильно), внутренняя задача может вообще не иметь минимума ($-\infty$) либо иметь его на целом множестве. Тогда двойственная $g(\nu)$ **не дифференцируема**.

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен,
только если лагранжиан имеет минимум по x .

- Если f **просто выпукла** (не сильно), внутренняя задача может вообще не иметь минимума ($-\infty$) либо иметь его на целом множестве. Тогда двойственная $g(\nu)$ **не дифференцируема**.

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен, только если лагранжиан имеет минимум по x .

- Если f **просто выпукла** (не сильно), внутренняя задача может вообще не иметь минимума ($-\infty$) либо иметь его на целом множестве. Тогда двойственная $g(\nu)$ **не дифференцируема**.
- Итог: для линейной сходимости двойственного подъёма нужна сильная выпуклость f и липшицев градиент. Это **жёсткие** условия.

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен, только если лагранжиан имеет минимум по x .

- Если f **просто выпукла** (не сильно), внутренняя задача может вообще не иметь минимума ($-\infty$) либо иметь его на целом множестве. Тогда двойственная $g(\nu)$ **не дифференцируема**.
- Итог: для линейной сходимости двойственного подъёма нужна сильная выпуклость f и липшицев градиент. Это **жёсткие** условия.

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен, только если лагранжиан имеет минимум по x .

- Если f **просто выпукла** (не сильно), внутренняя задача может вообще не иметь минимума ($-\infty$) либо иметь его на целом множестве. Тогда двойственная $g(\nu)$ **не дифференцируема**.
- Итог: для линейной сходимости двойственного подъёма нужна сильная выпуклость f и липшицев градиент. Это **жёсткие** условия.

Идея ALM. Добавим к целевой функции штраф за нарушение ограничения

$$\frac{\rho}{2} \|Ax - b\|^2, \quad \rho > 0.$$

На допустимом множестве ($Ax = b$) штраф равен нулю, поэтому задача **эквивалентна** исходной.

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен, только если лагранжиан имеет минимум по x .

- Если f **просто выпукла** (не сильно), внутренняя задача может вообще не иметь минимума ($-\infty$) либо иметь его на целом множестве. Тогда двойственная $g(\nu)$ **не дифференцируема**.
- Итог: для линейной сходимости двойственного подъёма нужна сильная выпуклость f и липшицев градиент. Это **жёсткие** условия.

Идея ALM. Добавим к целевой функции штраф за нарушение ограничения

$$\frac{\rho}{2} \|Ax - b\|^2, \quad \rho > 0.$$

На допустимом множестве ($Ax = b$) штраф равен нулю, поэтому задача **эквивалентна** исходной. Вдали от $Ax = b$ штраф искривляет целевую функцию вверх: задача становится **сильно выпуклой** (при A полного столбцового ранга), даже если f была лишь выпуклой.

Зачем нам модифицированная функция Лагранжа?

И двойственный подъём, и декомпозиция требуют сильной выпуклости f_i . Уберём это требование.

Проблема двойственного подъёма. Шаг

$x_k \in \arg \min_x [f(x) + \nu_{k-1}^T Ax]$ существует и единствен, только если лагранжиан имеет минимум по x .

- Если f **просто выпукла** (не сильно), внутренняя задача может вообще не иметь минимума ($-\infty$) либо иметь его на целом множестве. Тогда двойственная $g(\nu)$ **не дифференцируема**.
- Итог: для линейной сходимости двойственного подъёма нужна сильная выпуклость f и липшицев градиент. Это **жёсткие** условия.

Идея ALM. Добавим к целевой функции штраф за нарушение ограничения

$$\frac{\rho}{2} \|Ax - b\|^2, \quad \rho > 0.$$

На допустимом множестве ($Ax = b$) штраф равен нулю, поэтому задача **эквивалентна** исходной.

Вдали от $Ax = b$ штраф искривляет целевую функцию вверх: задача становится **сильно выпуклой** (при A полного столбцового ранга), даже если f была лишь выпуклой.

Цена: $\frac{\rho}{2} \|Ax - b\|^2$ связывает все координаты $x \Rightarrow$ теряем разделимость задачи.

Метод модифицированной функции Лагранжа

Метод модифицированной функции Лагранжа работает с прямой задачей в виде:

$$\min_x f(x) + \frac{\rho}{2} \|Ax - b\|^2$$

при $Ax = b$

Метод модифицированной функции Лагранжа

Метод модифицированной функции Лагранжа работает с прямой задачей в виде:

$$\min_x f(x) + \frac{\rho}{2} \|Ax - b\|^2$$

при $Ax = b$

где $\rho > 0$ — параметр. На допустимом множестве $Ax = b$ штраф равен нулю, поэтому задача эквивалентна исходной. Она сильно выпукла, если матрица A имеет полный столбцовый ранг.

Метод модифицированной функции Лагранжа

Метод модифицированной функции Лагранжа работает с прямой задачей в виде:

$$\min_x f(x) + \frac{\rho}{2} \|Ax - b\|^2$$

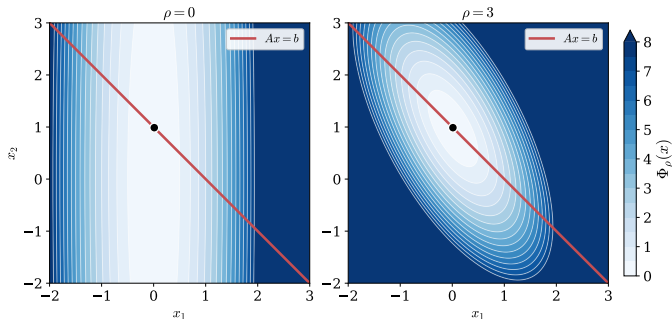
при $Ax = b$

где $\rho > 0$ — параметр. На допустимом множестве $Ax = b$ штраф равен нулю, поэтому задача эквивалентна исходной. Она сильно выпукла, если матрица A имеет полный столбцовый ранг.

Итерации ALM:

$$x_k = \arg \min_x \left[f(x) + (\nu_{k-1})^T Ax + \frac{\rho}{2} \|Ax - b\|^2 \right]$$
$$\nu_k = \nu_{k-1} + \rho(Ax_k - b)$$

Как штраф $\frac{\rho}{2}\|Ax - b\|^2$ делает задачу «более выпуклой»

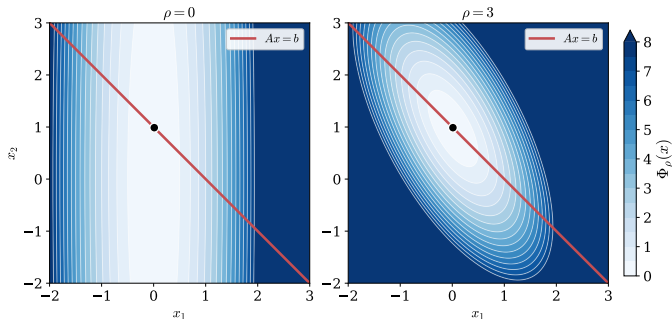


Модифицированная целевая функция

$$\Phi_{\rho}(x) = f(x) + \frac{\rho}{2}\|Ax - b\|^2.$$

Figure 12: Линии уровня $f(x) + \frac{\rho}{2}\|Ax - b\|^2$. Плоская вырожденная целевая функция (малая кривизна f вдоль собственного направления гессиана с собственным значением 0.05) превращается в выраженную чашу: штраф добавляет кривизну $\rho A^T A$ в направлениях, нарушающих $Ax = b$. Красным показана допустимая прямая $Ax = b$, оптимум на ней не двигается.

Как штраф $\frac{\rho}{2}\|Ax - b\|^2$ делает задачу «более выпуклой»



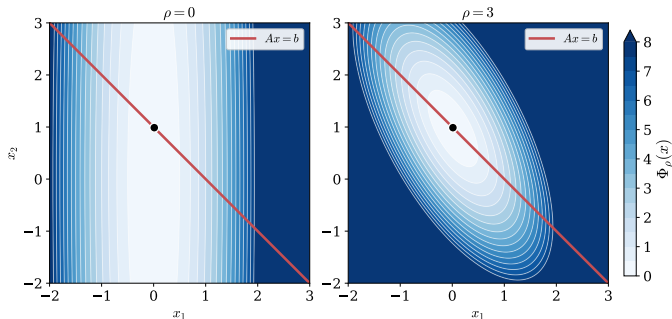
Модифицированная целевая функция

$$\Phi_{\rho}(x) = f(x) + \frac{\rho}{2}\|Ax - b\|^2.$$

- На множестве $Ax = b$ значение $\Phi_{\rho} = f$ **не меняется** при любом ρ , оптимум остаётся на месте.

Figure 12: Линии уровня $f(x) + \frac{\rho}{2}\|Ax - b\|^2$. Плоская вырожденная целевая функция (малая кривизна f вдоль собственного направления гессиана с собственным значением 0.05) превращается в выраженную чашу: штраф добавляет кривизну $\rho A^T A$ в направлениях, нарушающих $Ax = b$. Красным показана допустимая прямая $Ax = b$, оптимум на ней не двигается.

Как штраф $\frac{\rho}{2}\|Ax - b\|^2$ делает задачу «более выпуклой»



Модифицированная целевая функция

$$\Phi_{\rho}(x) = f(x) + \frac{\rho}{2}\|Ax - b\|^2.$$

- На множестве $Ax = b$ значение $\Phi_{\rho} = f$ **не меняется** при любом ρ , оптимум остаётся на месте.

Figure 12: Линии уровня $f(x) + \frac{\rho}{2}\|Ax - b\|^2$. Плоская вырожденная целевая функция (малая кривизна f вдоль собственного направления гессиана с собственным значением 0.05) превращается в выраженную чашу: штраф добавляет кривизну $\rho A^T A$ в направлениях, нарушающих $Ax = b$. Красным показана допустимая прямая $Ax = b$, оптимум на ней не двигается.

Как штраф $\frac{\rho}{2}\|Ax - b\|^2$ делает задачу «более выпуклой»

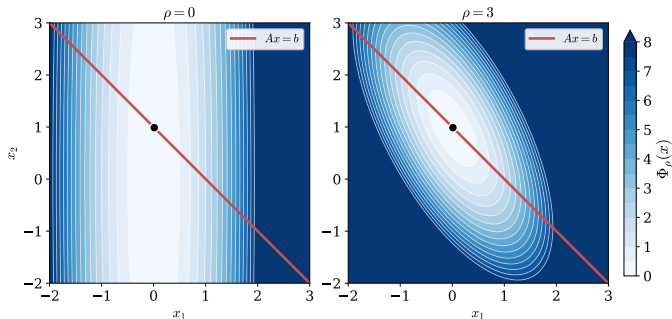


Figure 12: Линии уровня $f(x) + \frac{\rho}{2}\|Ax - b\|^2$. Плоская вырожденная целевая функция (малая кривизна f вдоль собственного направления гессиана с собственным значением 0.05) превращается в выраженную чашу: штраф добавляет кривизну $\rho A^T A$ в направлениях, нарушающих $Ax = b$. Красным показана допустимая прямая $Ax = b$, оптимум на ней не двигается.

Модифицированная целевая функция

$$\Phi_{\rho}(x) = f(x) + \frac{\rho}{2}\|Ax - b\|^2.$$

- На множестве $Ax = b$ значение $\Phi_{\rho} = f$ **не меняется** при любом ρ , оптимум остаётся на месте.
- В направлениях, нарушающих ограничение, добавляется кривизна $\rho A^T A$:

$$\mu_{\text{eff}} \gtrsim \rho \sigma_{\min}^2(A).$$

Как штраф $\frac{\rho}{2}\|Ax - b\|^2$ делает задачу «более выпуклой»

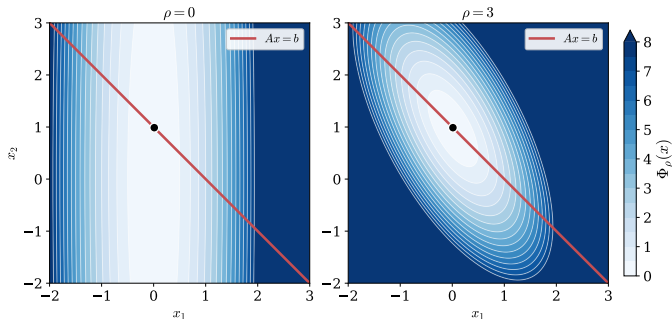


Figure 12: Линии уровня $f(x) + \frac{\rho}{2}\|Ax - b\|^2$. Плоская вырожденная целевая функция (малая кривизна f вдоль собственного направления гессиана с собственным значением 0.05) превращается в выраженную чашу: штраф добавляет кривизну $\rho A^T A$ в направлениях, нарушающих $Ax = b$. Красным показана допустимая прямая $Ax = b$, оптимум на ней не двигается.

Модифицированная целевая функция

$$\Phi_{\rho}(x) = f(x) + \frac{\rho}{2}\|Ax - b\|^2.$$

- На множестве $Ax = b$ значение $\Phi_{\rho} = f$ **не меняется** при любом ρ , оптимум остаётся на месте.
- В направлениях, нарушающих ограничение, добавляется кривизна $\rho A^T A$:

$$\mu_{\text{eff}} \gtrsim \rho \sigma_{\min}^2(A).$$

Как штраф $\frac{\rho}{2}\|Ax - b\|^2$ делает задачу «более выпуклой»

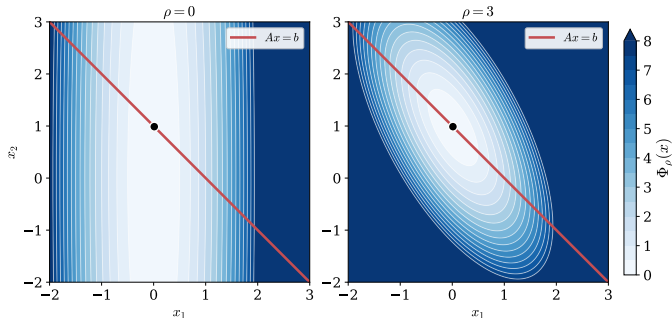


Figure 12: Линии уровня $f(x) + \frac{\rho}{2}\|Ax - b\|^2$. Плоская вырожденная целевая функция (малая кривизна f вдоль собственного направления гессиана с собственным значением 0.05) превращается в выраженную чашу: штраф добавляет кривизну $\rho A^T A$ в направлениях, нарушающих $Ax = b$. Красным показана допустимая прямая $Ax = b$, оптимум на ней не двигается.

Модифицированная целевая функция

$$\Phi_{\rho}(x) = f(x) + \frac{\rho}{2}\|Ax - b\|^2.$$

- На множестве $Ax = b$ значение $\Phi_{\rho} = f$ **не меняется** при любом ρ , оптимум остаётся на месте.
- В направлениях, нарушающих ограничение, добавляется кривизна $\rho A^T A$:

$$\mu_{\text{eff}} \gtrsim \rho \sigma_{\min}^2(A).$$

- Больше кривизны $\Rightarrow$ круглее линии уровня $\Rightarrow$ лучше обусловленность $\Rightarrow$ быстрее.

Почему шаг ровно $\alpha_k = \rho$?

Поскольку x_k минимизирует $f(x) + (\nu_{k-1})^T Ax + \frac{\rho}{2} \|Ax - b\|^2$ по x , имеем условие стационарности:

$$0 \in \partial f(x_k) + A^T \underbrace{(\nu_{k-1} + \rho(Ax_k - b))}_{=:\nu_k}$$

которое при выборе $\nu_k = \nu_{k-1} + \rho(Ax_k - b)$ (то есть $\alpha_k = \rho$) упрощается до:

$$0 \in \partial f(x_k) + A^T \nu_k$$

Почему шаг ровно $\alpha_k = \rho$?

Поскольку x_k минимизирует $f(x) + (\nu_{k-1})^T Ax + \frac{\rho}{2} \|Ax - b\|^2$ по x , имеем условие стационарности:

$$0 \in \partial f(x_k) + A^T \underbrace{(\nu_{k-1} + \rho(Ax_k - b))}_{=:\nu_k}$$

которое при выборе $\nu_k = \nu_{k-1} + \rho(Ax_k - b)$ (то есть $\alpha_k = \rho$) упрощается до:

$$0 \in \partial f(x_k) + A^T \nu_k$$

Это условие стационарности для исходной прямой задачи; при мягких условиях $Ax_k - b \rightarrow 0$, поэтому ККТ выполняются в пределе, а x_k, ν_k сходятся к решениям.

Почему шаг ровно $\alpha_k = \rho$?

Поскольку x_k минимизирует $f(x) + (\nu_{k-1})^T Ax + \frac{\rho}{2} \|Ax - b\|^2$ по x , имеем условие стационарности:

$$0 \in \partial f(x_k) + A^T \underbrace{(\nu_{k-1} + \rho(Ax_k - b))}_{=:\nu_k}$$

которое при выборе $\nu_k = \nu_{k-1} + \rho(Ax_k - b)$ (то есть $\alpha_k = \rho$) упрощается до:

$$0 \in \partial f(x_k) + A^T \nu_k$$

Это условие стационарности для исходной прямой задачи; при мягких условиях $Ax_k - b \rightarrow 0$, поэтому ККТ выполняются в пределе, а x_k, ν_k сходятся к решениям.

- Каждая итерация уже удовлетворяет стационарности по x ; осталось добиться допустимости, к чему и приводит штраф.

Почему шаг ровно $\alpha_k = \rho$?

Поскольку x_k минимизирует $f(x) + (\nu_{k-1})^T Ax + \frac{\rho}{2} \|Ax - b\|^2$ по x , имеем условие стационарности:

$$0 \in \partial f(x_k) + A^T \underbrace{(\nu_{k-1} + \rho(Ax_k - b))}_{=:\nu_k}$$

которое при выборе $\nu_k = \nu_{k-1} + \rho(Ax_k - b)$ (то есть $\alpha_k = \rho$) упрощается до:

$$0 \in \partial f(x_k) + A^T \nu_k$$

Это условие стационарности для исходной прямой задачи; при мягких условиях $Ax_k - b \rightarrow 0$, поэтому ККТ выполняются в пределе, а x_k, ν_k сходятся к решениям.

- Каждая итерация уже удовлетворяет стационарности по x ; осталось добиться допустимости, к чему и приводит штраф.
- При любом $\alpha \neq \rho$ это свойство теряется, поэтому $\alpha = \rho$ есть **следствие** структуры, а не подбор.

Почему шаг ровно $\alpha_k = \rho$?

Поскольку x_k минимизирует $f(x) + (\nu_{k-1})^T Ax + \frac{\rho}{2} \|Ax - b\|^2$ по x , имеем условие стационарности:

$$0 \in \partial f(x_k) + A^T \underbrace{(\nu_{k-1} + \rho(Ax_k - b))}_{=:\nu_k}$$

которое при выборе $\nu_k = \nu_{k-1} + \rho(Ax_k - b)$ (то есть $\alpha_k = \rho$) упрощается до:

$$0 \in \partial f(x_k) + A^T \nu_k$$

Это условие стационарности для исходной прямой задачи; при мягких условиях $Ax_k - b \rightarrow 0$, поэтому ККТ выполняются в пределе, а x_k, ν_k сходятся к решениям.

- Каждая итерация уже удовлетворяет стационарности по x ; осталось добиться допустимости, к чему и приводит штраф.
- При любом $\alpha \neq \rho$ это свойство теряется, поэтому $\alpha = \rho$ есть **следствие** структуры, а не подбор.
- Раскрывая $\frac{\rho}{2} \sum_{ij} x_i^T A_i^T A_j x_j$, видим перекрёстные члены, связывающие блоки; отсюда **потеря сепарабельности** (переход к ADMM).

Двойственный градиентный подъём и ALM: одна задача, два режима

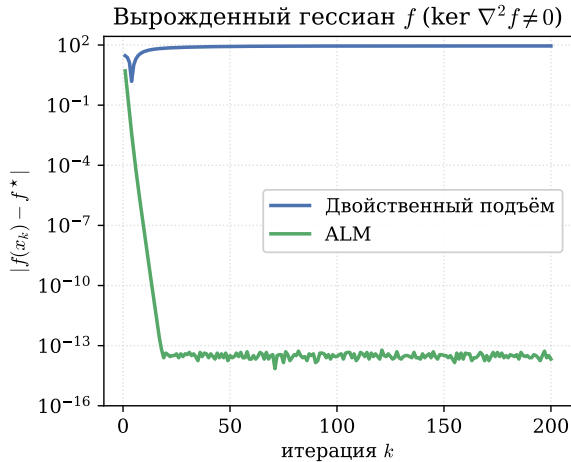
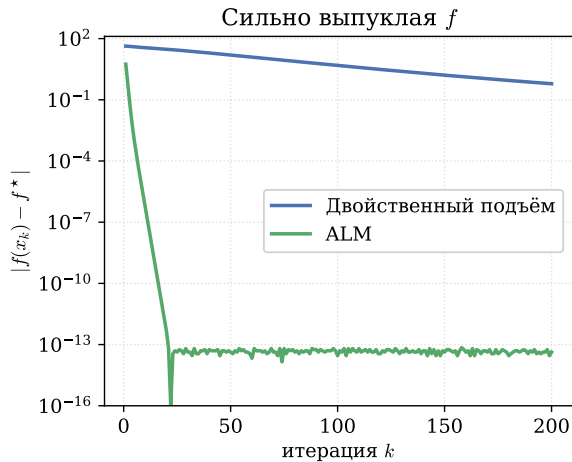


Figure 13: Квадратичная задача $Ax = b$. Слева (f сильно выпукла) сходятся оба метода, ALM быстрее. Справа (вырожденный гессиан): x -шаг двойственного подъёма не единствен, двойственная g негладкая, зазор не убывает; ALM сходится линейно за счёт штрафа.

Двойственный градиентный подъём и ALM: одна задача, два режима

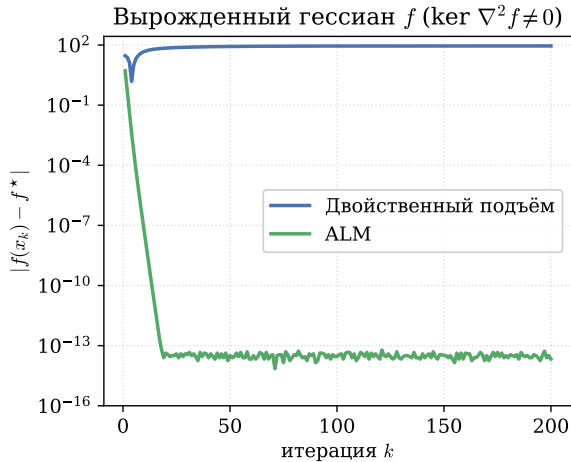
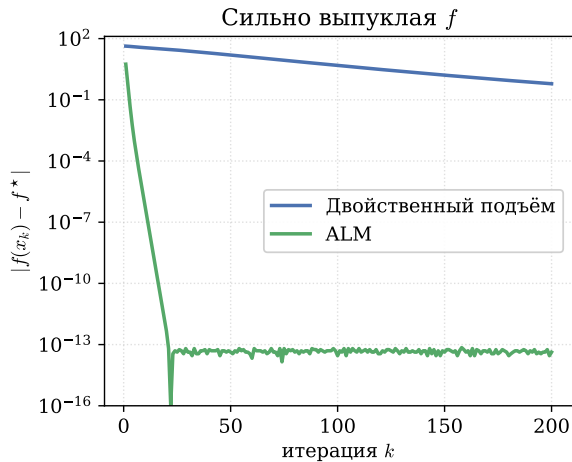


Figure 13: Квадратичная задача $Ax = b$. Слева (f сильно выпукла) сходятся оба метода, ALM быстрее. Справа (вырожденный гессиан): x -шаг двойственного подъёма не единствен, двойственная g негладкая, зазор не убывает; ALM сходится линейно за счёт штрафа.

Как выбирать ρ : компромисс

Компромисс выбора ρ в методе ALM

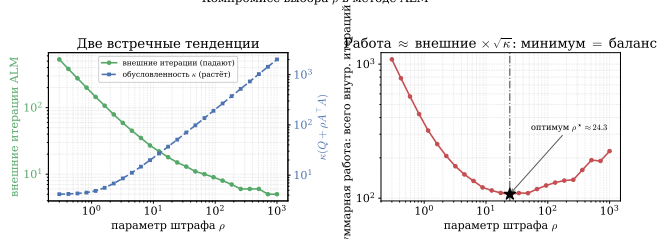


Figure 14: Слева: с ростом ρ внешних итераций ALM меньше (зелёная), но внутренняя задача $\min_x L_\rho$ хуже обусловлена (синяя), поэтому каждое внутреннее решение дороже. Справа: «суммарная работа» — это сколько всего внутренних итераций потратит метод = (число внешних итераций) $\times$ (стоимость одного внутреннего решения $\approx \sqrt{\kappa}$, столько шагов нужно градиентному методу при обусловленности κ). У произведения есть минимум — он и задаёт оптимальный ρ^* .

Большой ρ :

- быстрее $Ax_k \rightarrow b$, меньше внешних итераций;

Как выбирать ρ : компромисс

Компромисс выбора ρ в методе ALM

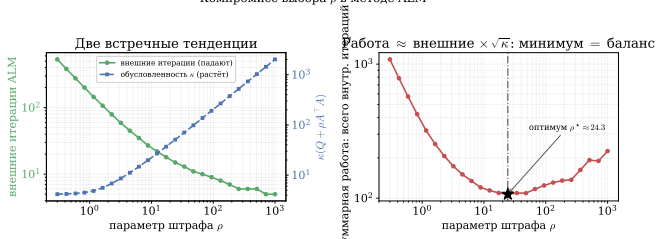


Figure 14: Слева: с ростом ρ внешних итераций ALM меньше (зелёная), но внутренняя задача $\min_x L_\rho$ хуже обусловлена (синяя), поэтому каждое внутреннее решение дороже. Справа: «суммарная работа» — это сколько всего внутренних итераций потратит метод = (число внешних итераций) $\times$ (стоимость одного внутреннего решения $\approx \sqrt{\kappa}$, столько шагов нужно градиентному методу при обусловленности κ). У произведения есть минимум — он и задаёт оптимальный ρ^* .

Большой ρ :

- быстрее $Ax_k \rightarrow b$, меньше внешних итераций;
- внутренний гессиан $\nabla^2 f + \rho A^T A$ хуже обусловлен, поэтому дороже.

Как выбирать ρ : компромисс

Компромисс выбора ρ в методе ALM

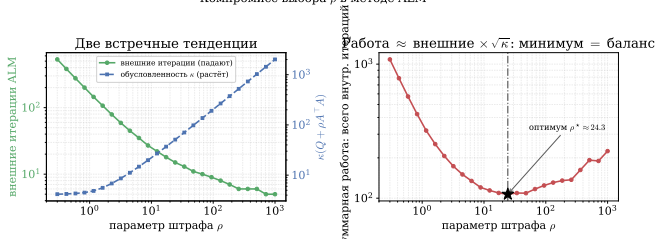


Figure 14: Слева: с ростом ρ внешних итераций ALM меньше (зелёная), но внутренняя задача $\min_x L_\rho$ хуже обусловлена (синяя), поэтому каждое внутреннее решение дороже. Справа: «суммарная работа» — это сколько всего внутренних итераций потратит метод = (число внешних итераций) $\times$ (стоимость одного внутреннего решения $\approx \sqrt{\kappa}$, столько шагов нужно градиентному методу при обусловленности κ). У произведения есть минимум — он и задаёт оптимальный ρ^* .

Большой ρ :

- быстрее $Ax_k \rightarrow b$, меньше внешних итераций;
- внутренний гессиан $\nabla^2 f + \rho A^T A$ хуже обусловлен, поэтому дороже.

Как выбирать ρ : компромисс

Компромисс выбора ρ в методе ALM

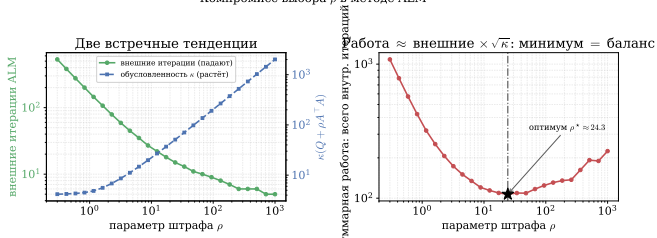


Figure 14: Слева: с ростом ρ внешних итераций ALM меньше (зелёная), но внутренняя задача $\min_x L_\rho$ хуже обусловлена (синяя), поэтому каждое внутреннее решение дороже. Справа: «суммарная работа» — это сколько всего внутренних итераций потратит метод = (число внешних итераций) $\times$ (стоимость одного внутреннего решения $\approx \sqrt{\kappa}$, столько шагов нужно градиентному методу при обусловленности κ). У произведения есть минимум — он и задаёт оптимальный ρ^* .

Большой ρ :

- быстрее $Ax_k \rightarrow b$, меньше внешних итераций;
- внутренний гессиан $\nabla^2 f + \rho A^T A$ хуже обусловлен, поэтому дороже.

Малый ρ :

- дешёвый, устойчивый внутренний шаг;

Как выбирать ρ : компромисс

Компромисс выбора ρ в методе ALM

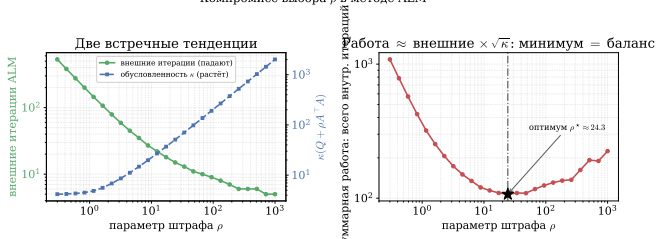


Figure 14: Слева: с ростом ρ внешних итераций ALM меньше (зелёная), но внутренняя задача $\min_x L_\rho$ хуже обусловлена (синяя), поэтому каждое внутреннее решение дороже. Справа: «суммарная работа» — это сколько всего внутренних итераций потратит метод = (число внешних итераций) $\times$ (стоимость одного внутреннего решения $\approx \sqrt{\kappa}$, столько шагов нужно градиентному методу при обусловленности κ). У произведения есть минимум — он и задаёт оптимальный ρ^* .

Большой ρ :

- быстрее $Ax_k \rightarrow b$, меньше внешних итераций;
- внутренний гессиан $\nabla^2 f + \rho A^T A$ хуже обусловлен, поэтому дороже.

Малый ρ :

- дешёвый, устойчивый внутренний шаг;
- медленная внешняя сходимость.

Как выбирать ρ : компромисс

Компромисс выбора ρ в методе ALM

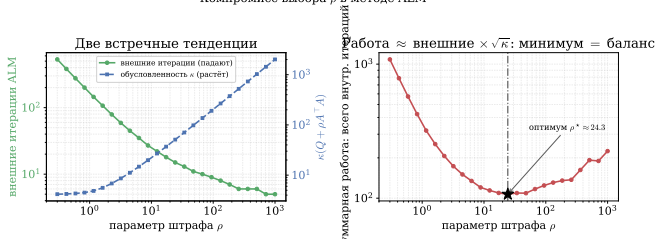


Figure 14: Слева: с ростом ρ внешних итераций ALM меньше (зелёная), но внутренняя задача $\min_x L_\rho$ хуже обусловлена (синяя), поэтому каждое внутреннее решение дороже. Справа: «суммарная работа» — это сколько всего внутренних итераций потратит метод = (число внешних итераций) $\times$ (стоимость одного внутреннего решения $\approx \sqrt{\kappa}$, столько шагов нужно градиентному методу при обусловленности κ). У произведения есть минимум — он и задаёт оптимальный ρ^* .

Большой ρ :

- быстрее $Ax_k \rightarrow b$, меньше внешних итераций;
- внутренний гессиан $\nabla^2 f + \rho A^T A$ хуже обусловлен, поэтому дороже.

Малый ρ :

- дешёвый, устойчивый внутренний шаг;
- медленная внешняя сходимость.

Как выбирать ρ : компромисс

Компромисс выбора ρ в методе ALM

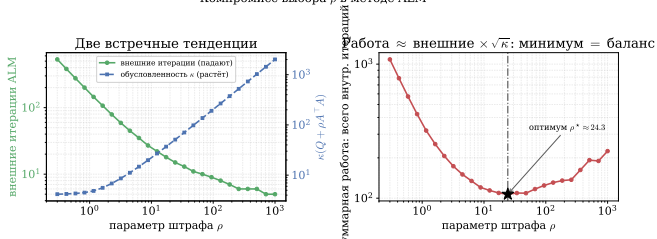


Figure 14: Слева: с ростом ρ внешних итераций ALM меньше (зелёная), но внутренняя задача $\min_x L_\rho$ хуже обусловлена (синяя), поэтому каждое внутреннее решение дороже. Справа: «суммарная работа» — это сколько всего внутренних итераций потратит метод = (число внешних итераций) $\times$ (стоимость одного внутреннего решения $\approx \sqrt{\kappa}$, столько шагов нужно градиентному методу при обусловленности κ). У произведения есть минимум — он и задаёт оптимальный ρ^* .

Большой ρ :

- быстрее $Ax_k \rightarrow b$, меньше внешних итераций;
- внутренний гессиан $\nabla^2 f + \rho A^T A$ хуже обусловлен, поэтому дороже.

Малый ρ :

- дешёвый, устойчивый внутренний шаг;
- медленная внешняя сходимость.

На практике: умеренный фиксированный ρ либо адаптивная схема, в которой ρ увеличивают, пока невязка $\|Ax_k - b\|$ убывает медленно (балансировка невязок, Boyd 2011).

Цена надёжности: ALM теряет декомпозицию

Пусть задача сепарабельна: $f(x) = \sum_i f_i(x_i)$ при $\sum_i A_i x_i = b$.

Цена надёжности: ALM теряет декомпозицию

Пусть задача сепарабельна: $f(x) = \sum_i f_i(x_i)$ при $\sum_i A_i x_i = b$.

В **двойственном подъёме** на каждом шаге минимизируем $\sum_i f_i(x_i) + \lambda^\top \sum_i A_i x_i$. Связь линейна по x , поэтому минимум распадается на независимые блоки x_i — это и есть декомпозиция.

Цена надёжности: ALM теряет декомпозицию

Пусть задача сепарабельна: $f(x) = \sum_i f_i(x_i)$ при $\sum_i A_i x_i = b$.

В **двойственном подъёме** на каждом шаге минимизируем $\sum_i f_i(x_i) + \lambda^\top \sum_i A_i x_i$. Связь линейна по x , поэтому минимум распадается на независимые блоки x_i — это и есть декомпозиция.

ALM добавляет квадратичный штраф. Раскроем его:

$$\frac{\rho}{2} \left\| \sum_i A_i x_i - b \right\|^2 = \frac{\rho}{2} \sum_{i,j} (A_i x_i)^\top (A_j x_j) - \rho b^\top \sum_i A_i x_i + \frac{\rho}{2} \|b\|^2.$$

Перекрёстные слагаемые при $i \neq j$ связывают блоки между собой: минимум по x больше не распадается.

Цена надёжности: ALM теряет декомпозицию

Пусть задача сепарабельна: $f(x) = \sum_i f_i(x_i)$ при $\sum_i A_i x_i = b$.

В **двойственном подъёме** на каждом шаге минимизируем $\sum_i f_i(x_i) + \lambda^\top \sum_i A_i x_i$. Связь линейна по x , поэтому минимум распадается на независимые блоки x_i — это и есть декомпозиция.

ALM добавляет квадратичный штраф. Раскроем его:

$$\frac{\rho}{2} \left\| \sum_i A_i x_i - b \right\|^2 = \frac{\rho}{2} \sum_{i,j} (A_i x_i)^\top (A_j x_j) - \rho b^\top \sum_i A_i x_i + \frac{\rho}{2} \|b\|^2.$$

Перекрёстные слагаемые при $i \neq j$ связывают блоки между собой: минимум по x больше не распадается.

В этом и разрыв: ALM устойчив, но монолитен. ADMM вернёт декомпозицию, обновляя блоки по очереди.

ADMM

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный
подъём

- **Разбивает** задачу на блоки
(параллельно по x_i).

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.
- Расщепляет гладкое и негладкое через x, z, ν .

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.
- Расщепляет гладкое и негладкое через x, z, ν .

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- Разбивает задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.
- Расщепляет гладкое и негладкое через x, z, ν .

Основная идея: вводим разделяющую переменную ($Ax + Bz = c$), чтобы f и g минимизировать **по отдельности**, и связываем их штрафом и множителем.

ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь $r(z)$ — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция $g(\nu)$ из прошлых разделов.

ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь $r(z)$ — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция $g(\nu)$ из прошлых разделов.

Добавим к целевой функции штраф за нарушение ограничения:

$$\min_{x,z} f(x) + r(z) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

$$\text{при } Ax + Bz = c$$

ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь $r(z)$ — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция $g(\nu)$ из прошлых разделов.

Добавим к целевой функции штраф за нарушение ограничения:

$$\min_{x,z} f(x) + r(z) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

$$\text{при } Ax + Bz = c$$

где $\rho > 0$ — параметр. Модифицированная функция Лагранжа:

$$L_\rho(x, z, \nu) = f(x) + r(z) + \nu^T (Ax + Bz - c) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь ν — множитель ограничения, шаг по нему идёт с коэффициентом ρ .

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь ν — множитель ограничения, шаг по нему идёт с коэффициентом ρ .

Отличие от ALM. Обычный ALM заменил бы первые два шага совместной минимизацией:

$$(x_k, z_k) = \arg \min_{x, z} L_\rho(x, z, \nu_{k-1})$$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь ν — множитель ограничения, шаг по нему идёт с коэффициентом ρ .

Отличие от ALM. Обычный ALM заменил бы первые два шага совместной минимизацией:

$$(x_k, z_k) = \arg \min_{x, z} L_\rho(x, z, \nu_{k-1})$$

ADMM **не** минимизирует (x, z) совместно, иначе переменные снова оказались бы связаны. Один проход по каждой переменной обходится чуть большим числом итераций, зато подзадачи f и r решаются по отдельности, часто в замкнутой форме.

Масштабированная форма: компактные шаги

Введём масштабированный множитель $u = \nu/\rho$: он прячет ρ внутрь нормы. Завершая квадрат, $\nu^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|^2 = \frac{\rho}{2}\|Ax + Bz - c + u\|^2 - \text{const.}$ Получаем компактную запись (исходный множитель $\nu = \rho u$):

$$x_k = \arg \min_x f(x) + \frac{\rho}{2}\|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2}\|Ax_k + Bz - c + u_{k-1}\|^2$$

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Масштабированная форма: компактные шаги

Введём масштабированный множитель $u = \nu/\rho$: он прячет ρ внутрь нормы. Завершая квадрат, $\nu^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|^2 = \frac{\rho}{2}\|Ax + Bz - c + u\|^2 - \text{const}$. Получаем компактную запись (исходный множитель $\nu = \rho u$):

$$x_k = \arg \min_x f(x) + \frac{\rho}{2}\|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2}\|Ax_k + Bz - c + u_{k-1}\|^2$$

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

В этой форме u -шаг идёт без ρ , поскольку её роль уже внутри нормы. Далее используем именно эту форму.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

- ρ играет роль жёсткости связи между x и z : при большом ρ согласование быстрее, но оптимизация каждой части медленнее.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

- ρ играет роль жёсткости связи между x и z : при большом ρ согласование быстрее, но оптимизация каждой части медленнее.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

- ρ играет роль жёсткости связи между x и z : при большом ρ согласование быстрее, но оптимизация каждой части медленнее.
- **Связь с прокс-методом.** При $B = I$ шаг z — это в точности проксимальный оператор: $z_k = \text{prox}_{r/\rho}(Ax_k + u_{k-1})$. ADMM обобщает проксимальный метод: прокс негладкой части r и шаг по гладкой f разнесены на разные переменные.

Невязки ADMM: критерий остановки и настройка ρ

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .
- $\|s_k\| \gg \|r_k\|$: решение слишком инертно $\Rightarrow$ **уменьшить ρ** .

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .
- $\|s_k\| \gg \|r_k\|$: решение слишком инертно $\Rightarrow$ **уменьшить ρ** .

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .
- $\|s_k\| \gg \|r_k\|$: решение слишком инертно $\Rightarrow$ **уменьшить ρ** .

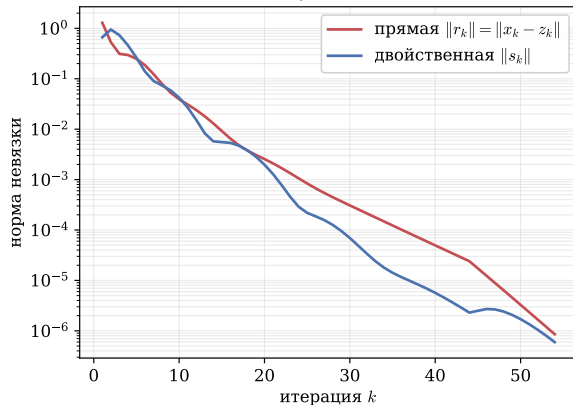
Адаптивное правило (Boyd et al., §3.4.1):

$$\rho_{k+1} = \begin{cases} 2\rho_k, & \|r_k\| > 10\|s_k\| \\ \rho_k/2, & \|s_k\| > 10\|r_k\| \\ \rho_k, & \text{иначе} \end{cases}$$

Простая эвристика, которая часто заметно ускоряет сходимость.

Невязки и адаптивный ρ : эксперимент

Адаптивный ρ : обе невязки $\rightarrow 0$



Цена неудачного ρ против адаптивного

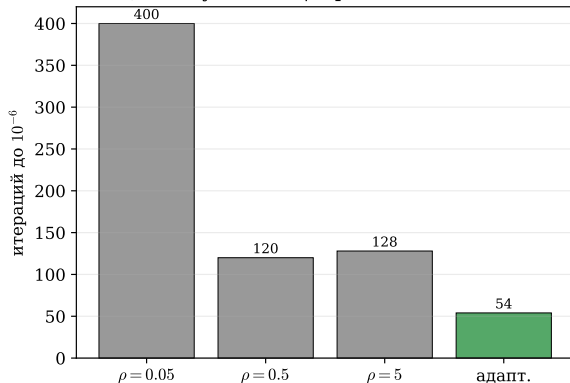


Figure 15: Слева: прямая и двойственная невязки ADMM на LASSO с адаптивным ρ , обе монотонно падают до 10^{-6} . Справа: число итераций до этой точности; неудачно выбранное фиксированное $\rho = 0.05$ заметно медленнее, а правило балансировки невязок сходится быстрее всех протестированных фиксированных ρ без ручного подбора (с одной рефакторизацией).

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.
- **Глобально (усреднение):** z есть **усреднение** локальных решений (и их множителей).

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.
- **Глобально (усреднение):** z есть **усреднение** локальных решений (и их множителей).

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.
- **Глобально (усреднение):** z есть **усреднение** локальных решений (и их множителей).

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Так ADMM применяют в федеративном обучении: данные не покидают узел, обмен идёт только векторами.

Консенсусный ADMM в действии

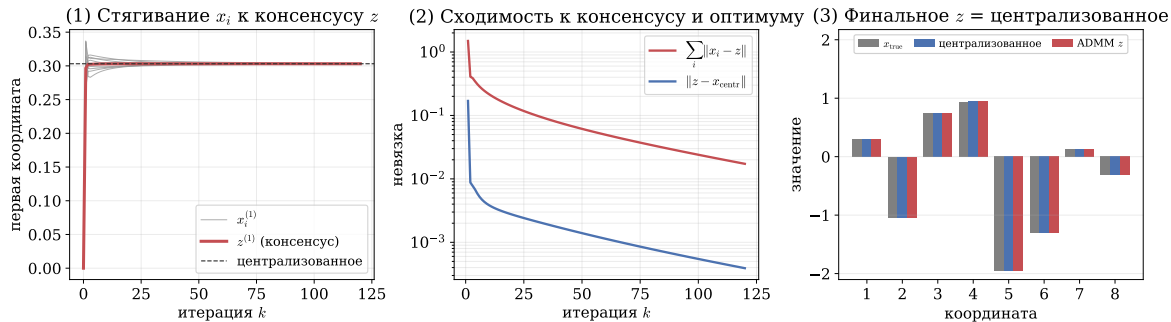


Figure 16: Распределённый МНК, $N = 10$ узлов. Слева: первые координаты локальных x_i (серые) стягиваются к консенсусу z (красная) и к централизованному решению. В центре: расхождение по координатам $\sum_i \|x_i - z\|$ и отклонение $\|z - x_{\text{centr}}\|$ от централизованного решения затухают. Справа: итоговое z совпадает с централизованным; по сети ходят только векторы.

Применения ADMM: один общий приём

За всеми применениями стоит один приём: негладкую часть r отдаём z -шагу, где prox_r имеет **замкнутую форму**; гладкую часть оставляем x -шагу.

Применения ADMM: один общий приём

За всеми применениями стоит один приём: негладкую часть r отдаём z -шагу, где prox_r имеет **замкнутую форму**; гладкую часть оставляем x -шагу.

$r(z)$	z -шаг (prox_r)	где применяется
$\lambda \ z\ _1$	мягкий порог	LASSO, разреженная регрессия
TV-штраф	покоординатное сжатие	TV-восстановление изображений
$\ Z\ _*$	усечение сингулярных чисел	robust PCA (низкий ранг)
$I_{\mathcal{C}}$	проекция на $\mathcal{C}$	конусные ограничения

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

Оговорка. На простом LASSO к высокой точности FISTA сопоставим или быстрее. ADMM выигрывает там, где есть несколько негладких частей, оператор внутри регуляризатора, жёсткие ограничения или распределённость (fused LASSO, TV, консенсус, robust PCA).

ADMM на практике: восстановление смазанного изображения

Смазано + шум (25.9 дБ)



ADMM (28.5 дБ)

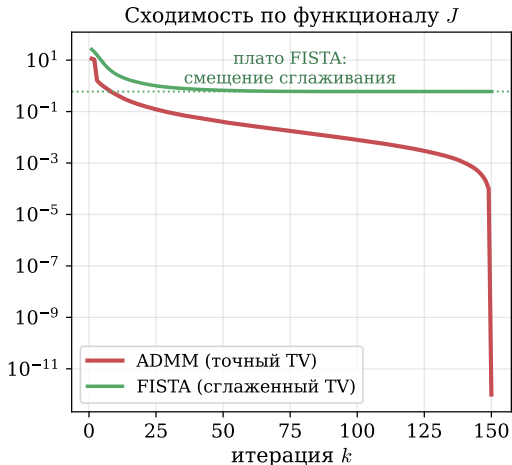


Figure 17: Устранение размытия настоящего фото. ADMM решает **точную** негладкую TV-задачу (FISTA минимизирует **сглаженный** суррогат и смещён по J). По качеству (PSNR) методы сопоставимы: FISTA 28.7 дБ, ADMM 28.5 дБ; ADMM ценен структурой, а не скоростью.

ADMM на практике: восстановление смазанного изображения

Смазано + шум (25.9 дБ)



ADMM (28.5 дБ)

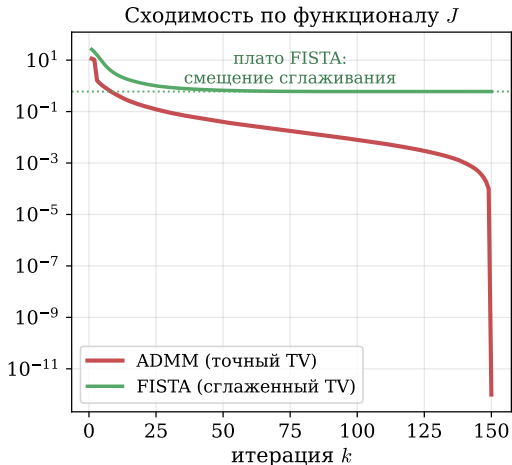


Figure 17: Устранение размытия настоящего фото. ADMM решает **точную** негладкую TV-задачу (FISTA минимизирует **сглаженный** суррогат и смещён по J). По качеству (PSNR) методы сопоставимы: FISTA 28.7 дБ, ADMM 28.5 дБ; ADMM ценен структурой, а не скоростью.

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 18: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 18: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 18: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);
- u -шаг: обновление множителя.

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 18: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);
- u -шаг: обновление множителя.

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



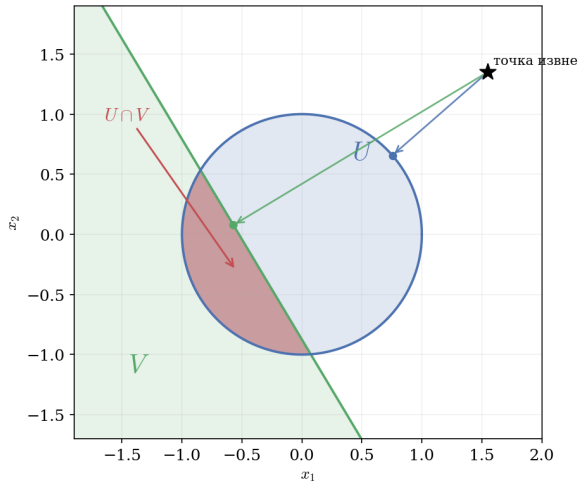
Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);
- u -шаг: обновление множителя.

Методы первого порядка не разбивают «ядерная норма + ℓ_1 » так естественно; здесь ADMM есть стандарт.

Figure 18: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Пример: метод альтернирующих проекций



Поиск точки в пересечении выпуклых множеств $U, V \subseteq \mathbb{R}^n$:

$$\min_x I_U(x) + I_V(x)$$

Приводим к форме ADMM ($A = I, B = -I, c = 0$):

$$\min_{x,z} I_U(x) + I_V(z) \quad \text{при} \quad x - z = 0$$

Каждый цикл ADMM включает две проекции:

$$x_k = P_U(z_{k-1} - u_{k-1})$$

$$z_k = P_V(x_k + u_{k-1})$$

$$u_k = u_{k-1} + x_k - z_k$$

Коррекция u даёт сходимость там, где простые поочерёдные проекции зацикливаются.

Figure 19: Сложную проекцию на пересечение $U \cap V$ (красный «серп») ADMM заменяет двумя простыми: на круг U и на полуплоскость V по отдельности.

Проекция на пересечение: ADMM в действии

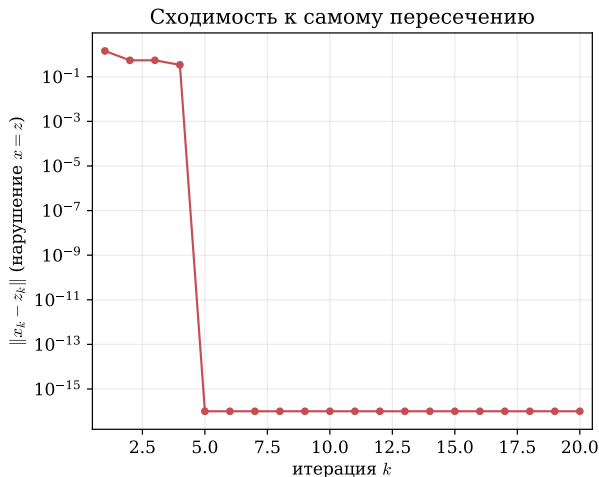
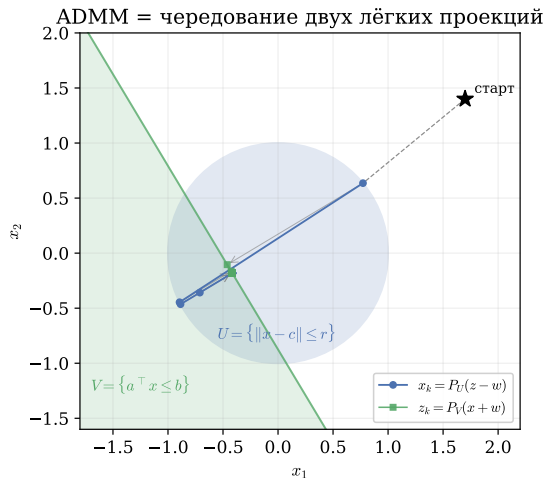


Figure 20: ADMM как чередующиеся проекции на круг U и полуплоскость V . Траектория зигзагом сходится в пересечение, $\|x_k - z_k\| \rightarrow 0$: сложную проекцию заменяем двумя простыми.

LASSO через ADMM: расщепить гладкое и негладкое

LASSO $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$: два слагаемых **разной природы**. Вводим копию $z = x$ и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

LASSO через ADMM: расщепить гладкое и негладкое

LASSO $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$: два слагаемых **разной природы**. Вводим копию $z = x$ и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

Шаги ADMM (масштабированная форма u):

$$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z_k - u_k)),$$

$$z_{k+1} = S_{\lambda/\rho}(x_{k+1} + u_k),$$

$$u_{k+1} = u_k + (x_{k+1} - z_{k+1}),$$

где $S_\tau(v) = \text{sign}(v) \cdot \max(|v| - \tau, 0)$ — мягкий порог.

	$\frac{1}{2} \ Ax - b\ ^2$	$\lambda \ z\ _1$
гладкость	да	нет
инструмент	система	мягкий порог
сложность	матрица A	покоординатно

LASSO через ADMM: расщепить гладкое и негладкое

LASSO $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$: два слагаемых **разной природы**. Вводим копию $z = x$ и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

Шаги ADMM (масштабированная форма u):

	$\frac{1}{2} \ Ax - b\ ^2$	$\lambda \ z\ _1$
гладкость	да	нет
инструмент	система	мягкий порог
сложность	матрица A	покоординатно

$$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho (z_k - u_k)),$$

$$z_{k+1} = S_{\lambda/\rho}(x_{k+1} + u_k),$$

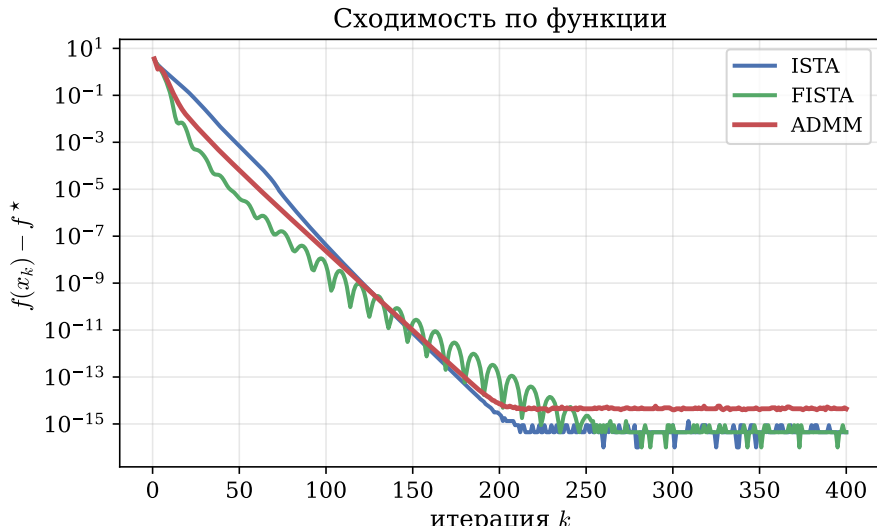
$$u_{k+1} = u_k + (x_{k+1} - z_{k+1}),$$

где $S_\tau(v) = \text{sign}(v) \cdot \max(|v| - \tau, 0)$ — мягкий порог.

x -шаг есть линейная система (естественная для квадратичной части), z -шаг есть мягкий порог в замкнутой форме (естественный для ℓ_1), u -шаг есть цена за рассогласование $x \neq z$. Факторизация $(A^T A + \rho I)$ считается один раз.

LASSO через ADMM: сравнение с ISTA / FISTA

LASSO: $m = 200$, $n = 500$, $\|x^*\|_0 = 25$



Где ADMM явно выигрывает: оператор в регуляризаторе

Возьмём **fused LASSO** $\min_x \frac{1}{2} \|x - y\|^2 + \lambda \|Dx\|_1$, где D — разностная матрица, $(Dx)_i = x_{i+1} - x_i$. У $\lambda \|Dx\|_1$ нет замкнутого прох (разность связывает координаты), поэтому прокс-градиент напрямую неприменим; прямой конкурент — субградиентный метод.

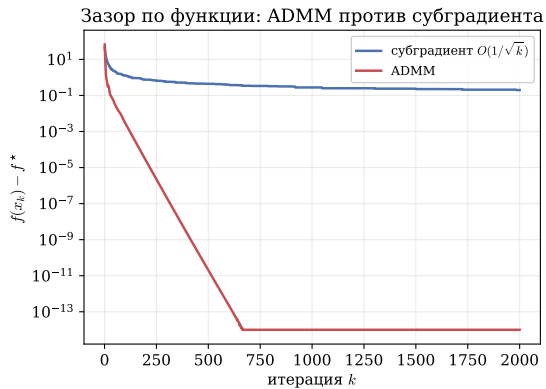
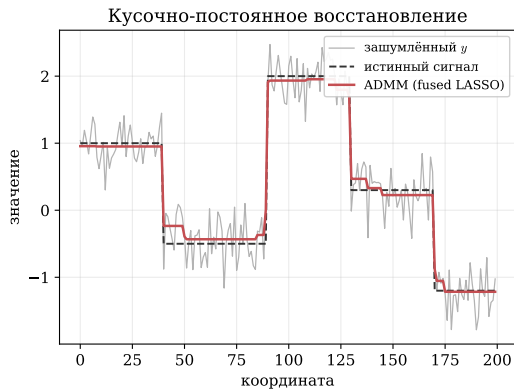


Figure 22: Слева: восстановление кусочно-постоянного сигнала из шума. Справа: ADMM ($z = Dx$) за ~ 700 итераций доходит до машинной точности, субградиент застревает на ~ 0.2 .

Двойственные методы и прежний инструментарий курса

«У меня уже есть прокс-градиент и проекция, зачем ADMM?» У каждого из прежних методов есть структура, на которой он перестаёт работать.

Метод курса	Где перестаёт работать	Что даёт ADMM
Субградиентный	негладкость, медленный $O(1/\sqrt{k})$	использует прох негладкой части (усадка)
Прокс-градиент / FISTA	прокс g сложен (TV с оператором, ≥ 2 слагаемых)	расщепляет на части с простым прох
Проекция градиента	проекция только на простые множества	пересечение множеств через расщепление
Франк–Вульф	составные негладкие целевые функции, распределённость	разбивает на блоки, цена связности есть множитель

Двойственные методы и прежний инструментарий курса

«У меня уже есть прокс-градиент и проекция, зачем ADMM?» У каждого из прежних методов есть структура, на которой он перестаёт работать.

Метод курса	Где перестаёт работать	Что даёт ADMM
Субградиентный	негладкость, медленный $O(1/\sqrt{k})$	использует прох негладкой части (усадка)
Прокс-градиент / FISTA	прокс g сложен (TV с оператором, ≥ 2 слагаемых)	расщепляет на части с простым прох
Проекция градиента	проекция только на простые множества	пересечение множеств через расщепление
Франк–Вульф	составные негладкие целевые функции, распределённость	разбивает на блоки, цена связности есть множитель

Прежние методы рассчитаны на **одну** удобную структуру; ADMM расщепляет задачу на части, каждую из которых они решают. На простой гладкой задаче ускоренные методы по-прежнему предпочтительнее.

Эволюция методов курса на LASSO

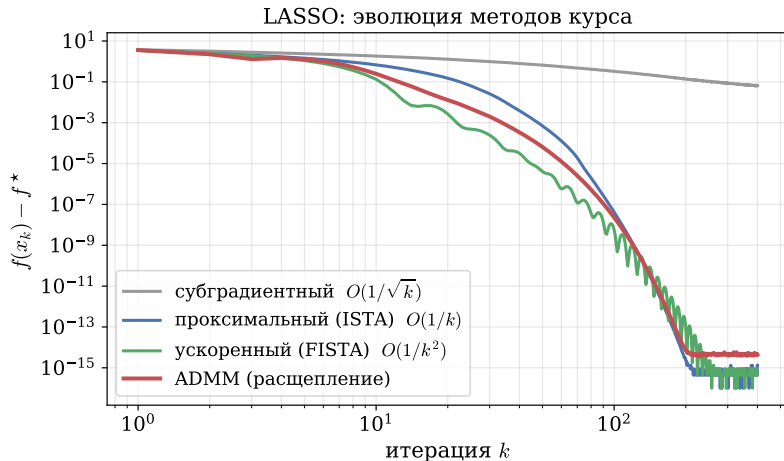


Figure 23: Субградиентный метод сходится медленно ($O(1/\sqrt{k})$) и застревает; проксимальный и ускоренный используют ргох негладкой части; ADMM расщепляет задачу. Именно ргох и расщепление есть причина перехода от субградиента к этим методам.

Какой метод когда выбирать: итог

Метод	Сильная выпуклость?	Параллельный?	Негладкая часть r ?	Скорость
Двойственный подъём	нужна	нет	через прокс	линейно (при μ -сильной)
Двойственная декомпозиция	нужна	да (B задач)	через прокс	как у двойственного подъёма
ALM	не нужна	нет	да	линейно, устойчив
ADMM	не нужна	да	да (z -шаг)	$O(1/k)$, на практике быстро

Какой метод когда выбирать: итог

Метод	Сильная выпуклость?	Параллельный?	Негладкая часть r ?	Скорость
Двойственный подъём	нужна	нет	через прокс	линейно (при μ -сильной)
Двойственная декомпозиция	нужна	да (B задач)	через прокс	как у двойственного подъёма
ALM	не нужна	нет	да	линейно, устойчив
ADMM	не нужна	да	да (z -шаг)	$O(1/k)$, на практике быстро

- Выбор метода сведён в таблицу выше; на практике чаще всего встречается случай «не сильно выпукло и есть негладкая часть» $\Rightarrow$ **ADMM**.

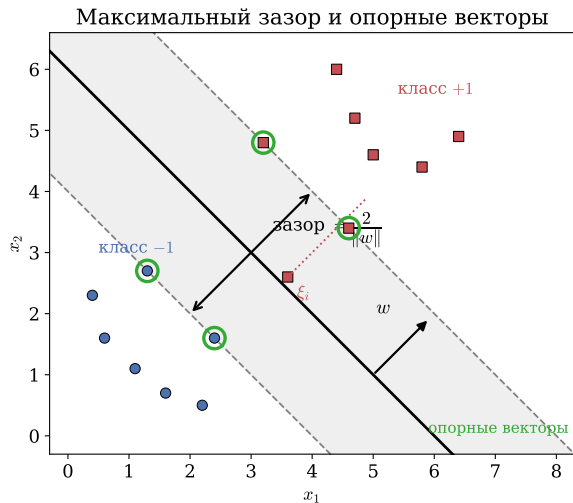
Какой метод когда выбирать: итог

Метод	Сильная выпуклость?	Параллельный?	Негладкая часть r ?	Скорость
Двойственный подъём	нужна	нет	через прокс	линейно (при μ -сильной)
Двойственная декомпозиция	нужна	да (B задач)	через прокс	как у двойственного подъёма
ALM	не нужна	нет	да	линейно, устойчив
ADMM	не нужна	да	да (z -шаг)	$O(1/k)$, на практике быстро

- Выбор метода сведён в таблицу выше; на практике чаще всего встречается случай «не сильно выпукло и есть негладкая часть» $\Rightarrow$ **ADMM**.
- ADMM применим широко, но требует подбора ρ и точного решения подзадач; на гладкой хорошо обусловленной задаче ускоренные методы быстрее.

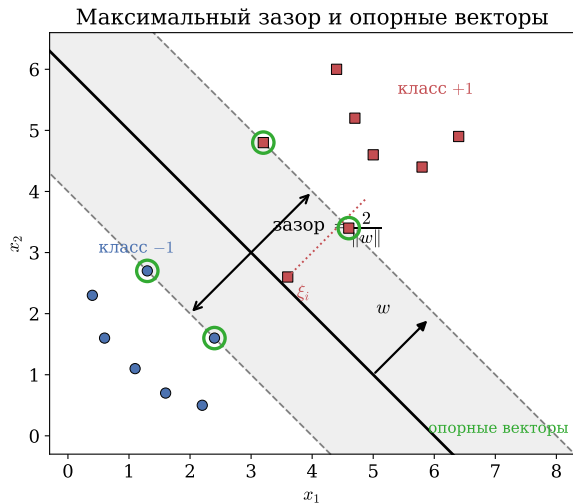
Возврат к примерам: откуда взялись те двойственные задачи

SVM: разделить классы с максимальным зазором



Разделяем два класса ($y_i \in \{\pm 1\}$) гиперплоскостью $w^\top x + b = 0$ так, чтобы **полоса** между ними была как можно шире.

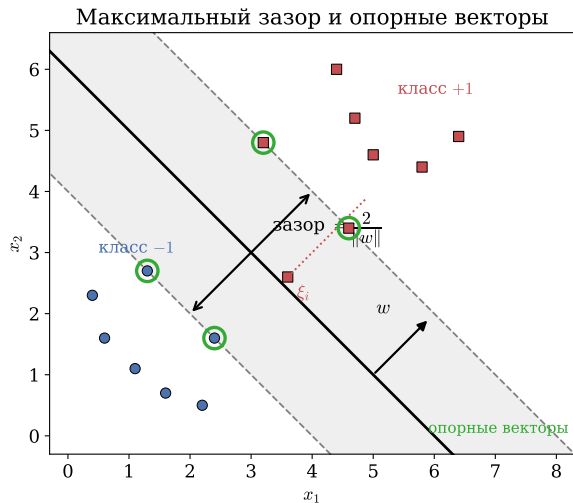
SVM: разделить классы с максимальным зазором



Разделяем два класса ($y_i \in \{\pm 1\}$) гиперплоскостью $w^\top x + b = 0$ так, чтобы **полоса** между ними была как можно шире.

- Ширина полосы (зазора) равна $2/\|w\|$ — чтобы расширить её, надо **минимизировать** $\|w\|$.

SVM: разделить классы с максимальным зазором

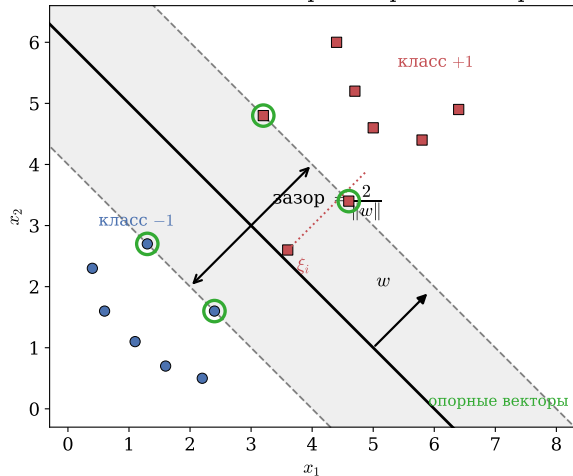


Разделяем два класса ($y_i \in \{\pm 1\}$) гиперплоскостью $w^\top x + b = 0$ так, чтобы **полоса** между ними была как можно шире.

- Ширина полосы (зазора) равна $2/\|w\|$ — чтобы расширить её, надо **минимизировать** $\|w\|$.
- На краях полосы лежат **опорные векторы** — только они «подпирают» границу.

SVM: разделить классы с максимальным зазором

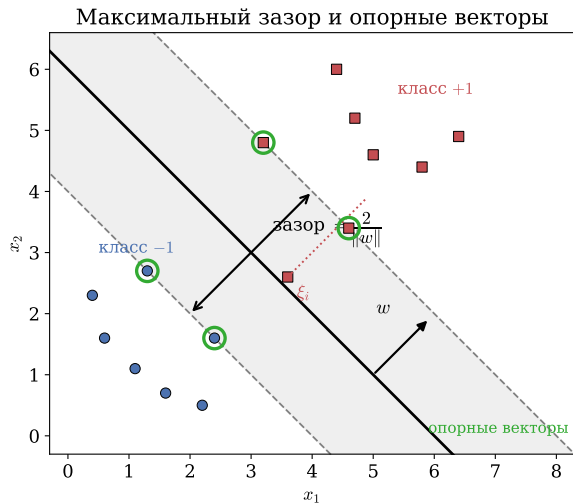
Максимальный зазор и опорные векторы



Разделяем два класса ($y_i \in \{\pm 1\}$) гиперплоскостью $w^\top x + b = 0$ так, чтобы **полоса** между ними была как можно шире.

- Ширина полосы (зазора) равна $2/\|w\|$ — чтобы расширить её, надо **минимизировать** $\|w\|$.
- На краях полосы лежат **опорные векторы** — только они «подпирают» границу.

SVM: разделить классы с максимальным зазором



Разделяем два класса ($y_i \in \{\pm 1\}$) гиперплоскостью $w^\top x + b = 0$ так, чтобы **полоса** между ними была как можно шире.

- Ширина полосы (зазора) равна $2/\|w\|$ — чтобы расширить её, надо **минимизировать** $\|w\|$.
- На краях полосы лежат **опорные векторы** — только они «подпирают» границу.

Данные не всегда разделимы. Разрешим нарушения: для точки i вводим **слабину** $\xi_i \geq 0$ — насколько она зашла за свой отступ — и штрафует $\sum_i \xi_i$.

SVM: прямая задача (мягкий отступ)

Собираем всё в одну задачу: широкий зазор $\Leftrightarrow$ малая $\|w\|$, плюс штраф C за суммарную слабину.

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{при} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

SVM: прямая задача (мягкий отступ)

Собираем всё в одну задачу: широкий зазор $\Leftrightarrow$ малая $\|w\|$, плюс штраф C за суммарную слабину.

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i \quad \text{при} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

- $y_i(w^\top x_i + b) \geq 1$ — точка по правильную сторону **за** отступом (уверенно классифицирована);

SVM: прямая задача (мягкий отступ)

Собираем всё в одну задачу: широкий зазор $\Leftrightarrow$ малая $\|w\|$, плюс штраф C за суммарную слабину.

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i \quad \text{при} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

- $y_i(w^\top x_i + b) \geq 1$ — точка по правильную сторону **за** отступом (уверенно классифицирована);
- $\xi_i > 0$ — точка зашла внутрь полосы или на чужую сторону (нарушитель);

SVM: прямая задача (мягкий отступ)

Собираем всё в одну задачу: широкий зазор $\Leftrightarrow$ малая $\|w\|$, плюс штраф C за суммарную слабину.

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{при} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

- $y_i(w^\top x_i + b) \geq 1$ — точка по правильную сторону **за** отступом (уверенно классифицирована);
- $\xi_i > 0$ — точка зашла внутрь полосы или на чужую сторону (нарушитель);
- C — цена нарушений: большой C почти запрещает их (жёсткий отступ), малый C расширяет полосу ценой нарушений.

SVM: прямая задача (мягкий отступ)

Собираем всё в одну задачу: широкий зазор $\Leftrightarrow$ малая $\|w\|$, плюс штраф C за суммарную слабину.

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{при} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

- $y_i(w^\top x_i + b) \geq 1$ — точка по правильную сторону **за** отступом (уверенно классифицирована);
- $\xi_i > 0$ — точка зашла внутрь полосы или на чужую сторону (нарушитель);
- C — цена нарушений: большой C почти запрещает их (жёсткий отступ), малый C расширяет полосу ценой нарушений.

SVM: прямая задача (мягкий отступ)

Собираем всё в одну задачу: широкий зазор $\Leftrightarrow$ малая $\|w\|$, плюс штраф C за суммарную слабинку.

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i \quad \text{при} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

- $y_i(w^\top x_i + b) \geq 1$ — точка по правильную сторону **за** отступом (уверенно классифицирована);
- $\xi_i > 0$ — точка зашла внутрь полосы или на чужую сторону (нарушитель);
- C — цена нарушений: большой C почти запрещает их (жёсткий отступ), малый C расширяет полосу ценой нарушений.

Проблема. Ограничения связывают w, b, ξ со всеми точками сразу — решать в таком виде неудобно. Перейдём к двойственной задаче.

SVM: Лагранжиан и условия стационарности

За каждое ограничение вводим **цену** (множитель): $\alpha_i \geq 0$ — за отступ, $\mu_i \geq 0$ — за $\xi_i \geq 0$. Лагранжиан:

$$L = \frac{1}{2}\|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(w^\top x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i.$$

SVM: Лагранжиан и условия стационарности

За каждое ограничение вводим **цену** (множитель): $\alpha_i \geq 0$ — за отступ, $\mu_i \geq 0$ — за $\xi_i \geq 0$. Лагранжиан:

$$L = \frac{1}{2}\|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(w^\top x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i.$$

Приравниваем производные по w, b, ξ к нулю:

$$w = \sum_i \alpha_i y_i x_i, \quad \sum_i \alpha_i y_i = 0, \quad \alpha_i + \mu_i = C.$$

SVM: Лагранжиан и условия стационарности

За каждое ограничение вводим **цену** (множитель): $\alpha_i \geq 0$ — за отступ, $\mu_i \geq 0$ — за $\xi_i \geq 0$. Лагранжиан:

$$L = \frac{1}{2}\|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(w^\top x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i.$$

Приравниваем производные по w, b, ξ к нулю:

$$w = \sum_i \alpha_i y_i x_i, \quad \sum_i \alpha_i y_i = 0, \quad \alpha_i + \mu_i = C.$$

- $w = \sum_i \alpha_i y_i x_i$: **разделяющая строится из самих точек**, веса — это α_i .

SVM: Лагранжиан и условия стационарности

За каждое ограничение вводим **цену** (множитель): $\alpha_i \geq 0$ — за отступ, $\mu_i \geq 0$ — за $\xi_i \geq 0$. Лагранжиан:

$$L = \frac{1}{2}\|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(w^\top x_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i.$$

Приравниваем производные по w, b, ξ к нулю:

$$w = \sum_i \alpha_i y_i x_i, \quad \sum_i \alpha_i y_i = 0, \quad \alpha_i + \mu_i = C.$$

- $w = \sum_i \alpha_i y_i x_i$: **разделяющая строится из самих точек**, веса — это α_i .
- из $\alpha_i + \mu_i = C$ и $\mu_i \geq 0$ следует **коробка** $0 \leq \alpha_i \leq C$.

SVM: двойственная задача и её смысл

Подставляем эти соотношения назад в L — переменные w, b, ξ исчезают, остаётся задача **только по** α :

$$\max_{0 \leq \alpha \leq C} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \underbrace{x_i^\top x_j}_{K(x_i, x_j)}, \quad \sum_i \alpha_i y_i = 0.$$

SVM: двойственная задача и её смысл

Подставляем эти соотношения назад в L — переменные w, b, ξ исчезают, остаётся задача **только по α** :

$$\max_{0 \leq \alpha \leq C} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \underbrace{x_i^\top x_j}_{K(x_i, x_j)}, \quad \sum_i \alpha_i y_i = 0.$$

Это выпуклая квадратичная задача по α — её и решают на практике.

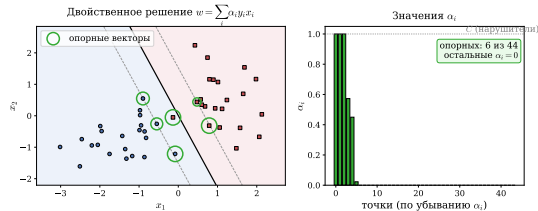


Figure 24: Решение двойственной: $\alpha_i > 0$ только у опорных векторов.

SVM: двойственная задача и её смысл

Подставляем эти соотношения назад в L — переменные w, b, ξ исчезают, остаётся задача **только по α** :

$$\max_{0 \leq \alpha \leq C} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \underbrace{x_i^\top x_j}_{K(x_i, x_j)}, \quad \sum_i \alpha_i y_i = 0.$$

Это выпуклая квадратичная задача по α — её и решают на практике.

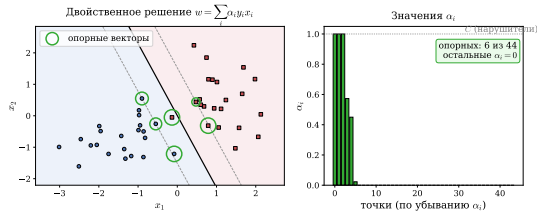


Figure 24: Решение двойственной: $\alpha_i > 0$ только у опорных векторов.

- **Разреженность.** $\alpha_i > 0$ лишь у опорных векторов — решение задаётся горсткой точек.

SVM: двойственная задача и её смысл

Подставляем эти соотношения назад в L — переменные w, b, ξ исчезают, остаётся задача **только по α** :

$$\max_{0 \leq \alpha \leq C} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \underbrace{x_i^\top x_j}_{K(x_i, x_j)}, \quad \sum_i \alpha_i y_i = 0.$$

Это выпуклая квадратичная задача по α — её и решают на практике.

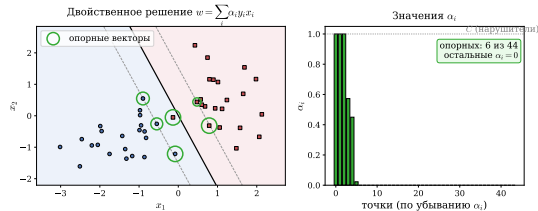


Figure 24: Решение двойственной: $\alpha_i > 0$ только у опорных векторов.

- **Разреженность.** $\alpha_i > 0$ лишь у опорных векторов — решение задаётся горсткой точек.
- **Ядра.** Данные входят только через $x_i^\top x_j \rightarrow$ замена на ядро $K(x_i, x_j)$ даёт нелинейный SVM, не меняя схему решения.

Заполнение водой: постановка

Передатчик делит мощность P между n каналами связи. У канала i усиление g_i ; вложив в него мощность x_i , получаем скорость $\log(1 + g_i x_i)$ (формула Шеннона).

Заполнение водой: постановка

Передатчик делит мощность P между n каналами связи. У канала i усиление g_i ; вложив в него мощность x_i , получаем скорость $\log(1 + g_i x_i)$ (формула Шеннона).

Распределяем бюджет P так, чтобы суммарная скорость была наибольшей:

$$\max_{x \succeq 0} \sum_i \log(1 + g_i x_i) \quad \text{при} \quad \sum_i x_i = P.$$

Заполнение водой: постановка

Передатчик делит мощность P между n каналами связи. У канала i усиление g_i ; вложив в него мощность x_i , получаем скорость $\log(1 + g_i x_i)$ (формула Шеннона).

Распределяем бюджет P так, чтобы суммарная скорость была наибольшей:

$$\max_{x \succeq 0} \sum_i \log(1 + g_i x_i) \quad \text{при} \quad \sum_i x_i = P.$$

- Каждое $\log(1 + g_i x_i)$ вогнуто и растёт с насыщением — отдача от добавления мощности падает.

Заполнение водой: постановка

Передатчик делит мощность P между n каналами связи. У канала i усиление g_i ; вложив в него мощность x_i , получаем скорость $\log(1 + g_i x_i)$ (формула Шеннона).

Распределяем бюджет P так, чтобы суммарная скорость была наибольшей:

$$\max_{x \succeq 0} \sum_i \log(1 + g_i x_i) \quad \text{при} \quad \sum_i x_i = P.$$

- Каждое $\log(1 + g_i x_i)$ вогнуто и растёт с насыщением — отдача от добавления мощности падает.
- Сильным каналам (большое g_i) выгоднее давать больше — но сколько именно? Ответ даст двойственная переменная.

Заполнение водой: вывод из условий ККТ

Вводим множитель ν за бюджет $\sum_i x_i = P$ и $\lambda_i \geq 0$ за $x_i \geq 0$. Стационарность по x_i :

$$\frac{g_i}{1 + g_i x_i} - \nu + \lambda_i = 0.$$

Заполнение водой: вывод из условий ККТ

Вводим множитель ν за бюджет $\sum_i x_i = P$ и $\lambda_i \geq 0$ за $x_i \geq 0$. Стационарность по x_i :

$$\frac{g_i}{1 + g_i x_i} - \nu + \lambda_i = 0.$$

Дополняющая нежёсткость $\lambda_i x_i = 0$ даёт два случая:

- канал **активен** ($x_i > 0$): тогда $\lambda_i = 0$ и $\frac{g_i}{1 + g_i x_i} = \nu \Rightarrow x_i = \frac{1}{\nu} - \frac{1}{g_i}$;

Заполнение водой: вывод из условий ККТ

Вводим множитель ν за бюджет $\sum_i x_i = P$ и $\lambda_i \geq 0$ за $x_i \geq 0$. Стационарность по x_i :

$$\frac{g_i}{1 + g_i x_i} - \nu + \lambda_i = 0.$$

Дополняющая нежесткость $\lambda_i x_i = 0$ даёт два случая:

- канал **активен** ($x_i > 0$): тогда $\lambda_i = 0$ и $\frac{g_i}{1 + g_i x_i} = \nu \Rightarrow x_i = \frac{1}{\nu} - \frac{1}{g_i}$;
- канал **выключен** ($x_i = 0$): это когда дно $1/g_i$ выше уровня $1/\nu$ (слабый канал).

Заполнение водой: вывод из условий ККТ

Вводим множитель ν за бюджет $\sum_i x_i = P$ и $\lambda_i \geq 0$ за $x_i \geq 0$. Стационарность по x_i :

$$\frac{g_i}{1 + g_i x_i} - \nu + \lambda_i = 0.$$

Дополняющая нежесткость $\lambda_i x_i = 0$ даёт два случая:

- канал **активен** ($x_i > 0$): тогда $\lambda_i = 0$ и $\frac{g_i}{1 + g_i x_i} = \nu \Rightarrow x_i = \frac{1}{\nu} - \frac{1}{g_i}$;
- канал **выключен** ($x_i = 0$): это когда дно $1/g_i$ выше уровня $1/\nu$ (слабый канал).

Заполнение водой: вывод из условий ККТ

Вводим множитель ν за бюджет $\sum_i x_i = P$ и $\lambda_i \geq 0$ за $x_i \geq 0$. Стационарность по x_i :

$$\frac{g_i}{1 + g_i x_i} - \nu + \lambda_i = 0.$$

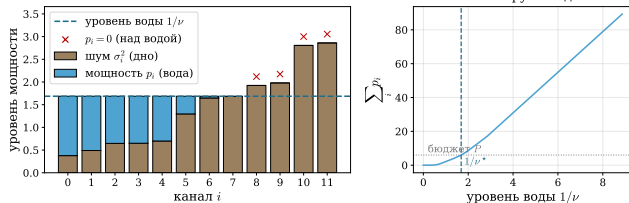
Дополняющая нежесткость $\lambda_i x_i = 0$ даёт два случая:

- канал **активен** ($x_i > 0$): тогда $\lambda_i = 0$ и $\frac{g_i}{1 + g_i x_i} = \nu \Rightarrow x_i = \frac{1}{\nu} - \frac{1}{g_i}$;
- канал **выключен** ($x_i = 0$): это когда дно $1/g_i$ выше уровня $1/\nu$ (слабый канал).

Объединяя оба случая, получаем замкнутую формулу:

$$x_i = \left(\frac{1}{\nu} - \frac{1}{g_i} \right)_+$$

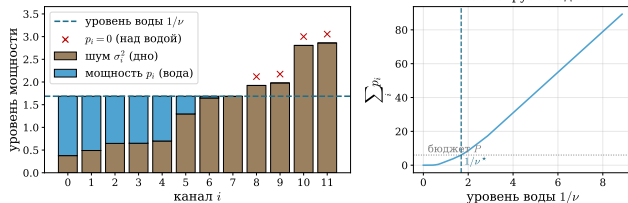
Заполнение водой: картинка и смысл ν



Формула $x_i = (1/\nu - 1/g_i)_+$ — это **ровно картинка заливки**:

Figure 25: «Дно» $1/g_i$ — шум канала; льём воду до единого уровня $1/\nu$; глубина над дном и есть мощность x_i .

Заполнение водой: картинка и смысл ν



Формула $x_i = (1/\nu - 1/g_i)_+$ — это **ровно картина заливки**:

- $1/g_i$ — высота «дна» канала (шум);

Figure 25: «Дно» $1/g_i$ — шум канала; льём воду до единого уровня $1/\nu$; глубина над дном и есть мощность x_i .

Заполнение водой: картинка и смысл ν

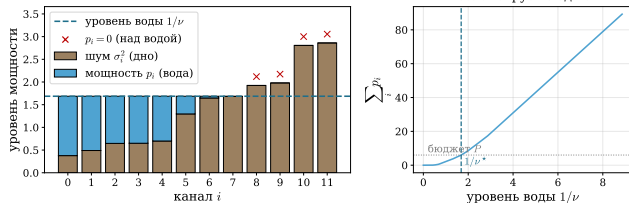


Figure 25: «Дно» $1/g_i$ — шум канала; льём воду до единого уровня $1/\nu$; глубина над дном и есть мощность x_i .

Формула $x_i = (1/\nu - 1/g_i)_+$ — это **ровно картина заливки**:

- $1/g_i$ — высота «дна» канала (шум);
- $1/\nu$ — единый «уровень воды»;

Заполнение водой: картинка и смысл ν

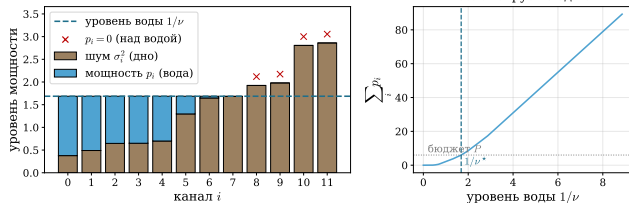


Figure 25: «Дно» $1/g_i$ — шум канала; льём воду до единого уровня $1/\nu$; глубина над дном и есть мощность x_i .

Формула $x_i = (1/\nu - 1/g_i)_+$ — это **ровно картина заливки**:

- $1/g_i$ — высота «дна» канала (шум);
- $1/\nu$ — единый «уровень воды»;
- x_i — глубина воды над дном;

Заполнение водой: картинка и смысл ν

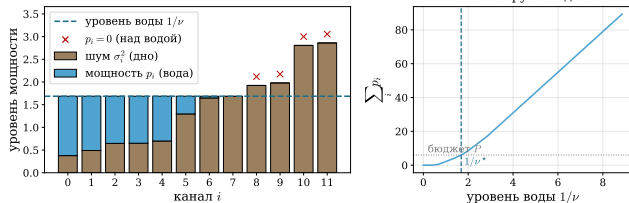


Figure 25: «Дно» $1/g_i$ — шум канала; льём воду до единого уровня $1/\nu$; глубина над дном и есть мощность x_i .

Формула $x_i = (1/\nu - 1/g_i)_+$ — это **ровно картина заливки**:

- $1/g_i$ — высота «дна» канала (шум);
- $1/\nu$ — единый «уровень воды»;
- x_i — глубина воды над дном;
- слабый канал ($1/g_i > 1/\nu$) остаётся над водой $\rightarrow x_i = 0$.

Заполнение водой: картинка и смысл ν

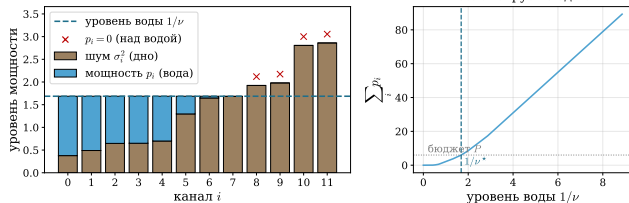


Figure 25: «Дно» $1/g_i$ — шум канала; льём воду до единого уровня $1/\nu$; глубина над дном и есть мощность x_i .

Формула $x_i = (1/\nu - 1/g_i)_+$ — это **ровно картина заливки**:

- $1/g_i$ — высота «дна» канала (шум);
- $1/\nu$ — единый «уровень воды»;
- x_i — глубина воды над дном;
- слабый канал ($1/g_i > 1/\nu$) остаётся над водой $\rightarrow x_i = 0$.

Заполнение водой: картинка и смысл ν

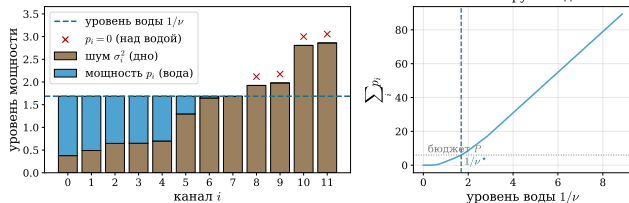


Figure 25: «Дно» $1/g_i$ — шум канала; льём воду до единого уровня $1/\nu$; глубина над дном и есть мощность x_i .

Формула $x_i = (1/\nu - 1/g_i)_+$ — это **ровно картина заливки**:

- $1/g_i$ — высота «дна» канала (шум);
- $1/\nu$ — единый «уровень воды»;
- x_i — глубина воды над дном;
- слабый канал ($1/g_i > 1/\nu$) остаётся над водой $\rightarrow x_i = 0$.

Один скаляр ν подбирается из бюджета $\sum_i x_i = P$ и задаёт раздачу сразу по всем каналам. Вот прямой физический смысл двойственной переменной.

- Ryan Tibshirani. Convex Optimization 10-725

- Ryan Tibshirani. Convex Optimization 10-725
- Boyd S., Vandenberghe L. *Convex Optimization*, Cambridge University Press, 2004 (гл. 5).

- Ryan Tibshirani. Convex Optimization 10-725
- Boyd S., Vandenberghe L. *Convex Optimization*, Cambridge University Press, 2004 (гл. 5).
- Boyd S., Parikh N., Chu E., Peleato B., Eckstein J. *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers*, Foundations and Trends in ML, 2011.

- Ryan Tibshirani. Convex Optimization 10-725
- Boyd S., Vandenberghe L. *Convex Optimization*, Cambridge University Press, 2004 (гл. 5).
- Boyd S., Parikh N., Chu E., Peleato B., Eckstein J. *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers*, Foundations and Trends in ML, 2011.
- Goldstein T., Osher S. *The Split Bregman Method for L1-Regularized Problems*, SIAM J. Imaging Sci., 2009 — TV-обработка изображений.

- Ryan Tibshirani. *Convex Optimization* 10-725
- Boyd S., Vandenberghe L. *Convex Optimization*, Cambridge University Press, 2004 (гл. 5).
- Boyd S., Parikh N., Chu E., Peleato B., Eckstein J. *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers*, Foundations and Trends in ML, 2011.
- Goldstein T., Osher S. *The Split Bregman Method for L1-Regularized Problems*, SIAM J. Imaging Sci., 2009 — TV-обработка изображений.
- Candès E., Li X., Ma Y., Wright J. *Robust Principal Component Analysis?*, J. ACM, 2011 — низкоранг + разреженность.

Приложение: доказательства

Свойства сопряжённой функции (доказательства)

Покажем, что $x \in \partial f^*(y) \Leftrightarrow y \in \partial f(x)$, предполагая, что f выпукла и замкнута.

- **Доказательство** $\Leftarrow$: Пусть $y \in \partial f(x)$. Тогда $x \in M_y$, где M_y — множество точек, в которых достигается максимум $y^T z - f(z)$ по z . Но

$$f^*(y) = \max_z \{y^T z - f(z)\}, \quad \partial f^*(y) = \text{cl conv} \left(\bigcup_{z \in M_y} \{z\} \right).$$

Поэтому $x \in \partial f^*(y)$.

Свойства сопряжённой функции (доказательства)

Покажем, что $x \in \partial f^*(y) \Leftrightarrow y \in \partial f(x)$, предполагая, что f выпукла и замкнута.

- **Доказательство** $\Leftarrow$: Пусть $y \in \partial f(x)$. Тогда $x \in M_y$, где M_y — множество точек, в которых достигается максимум $y^T z - f(z)$ по z . Но

$$f^*(y) = \max_z \{y^T z - f(z)\}, \quad \partial f^*(y) = \text{cl conv} \left(\bigcup_{z \in M_y} \{z\} \right).$$

Поэтому $x \in \partial f^*(y)$.

- **Доказательство** $\Rightarrow$: Применим уже показанное направление к функции f^* : если $x \in \partial f^*(y)$, то $y \in \partial f^{**}(x)$; а так как f замкнута и выпукла, $f^{**} = f$, поэтому $y \in \partial f(x)$.

Свойства сопряжённой функции (доказательства)

Покажем, что $x \in \partial f^*(y) \Leftrightarrow y \in \partial f(x)$, предполагая, что f выпукла и замкнута.

- **Доказательство** $\Leftarrow$: Пусть $y \in \partial f(x)$. Тогда $x \in M_y$, где M_y — множество точек, в которых достигается максимум $y^T z - f(z)$ по z . Но

$$f^*(y) = \max_z \{y^T z - f(z)\}, \quad \partial f^*(y) = \text{cl conv} \left(\bigcup_{z \in M_y} \{z\} \right).$$

Поэтому $x \in \partial f^*(y)$.

- **Доказательство** $\Rightarrow$: Применим уже показанное направление к функции f^* : если $x \in \partial f^*(y)$, то $y \in \partial f^{**}(x)$; а так как f замкнута и выпукла, $f^{**} = f$, поэтому $y \in \partial f(x)$.

Свойства сопряжённой функции (доказательства)

Покажем, что $x \in \partial f^*(y) \Leftrightarrow y \in \partial f(x)$, предполагая, что f выпукла и замкнута.

- **Доказательство** $\Leftarrow$: Пусть $y \in \partial f(x)$. Тогда $x \in M_y$, где M_y — множество точек, в которых достигается максимум $y^T z - f(z)$ по z . Но

$$f^*(y) = \max_z \{y^T z - f(z)\}, \quad \partial f^*(y) = \text{cl conv} \left(\bigcup_{z \in M_y} \{z\} \right).$$

Поэтому $x \in \partial f^*(y)$.

- **Доказательство** $\Rightarrow$: Применим уже показанное направление к функции f^* : если $x \in \partial f^*(y)$, то $y \in \partial f^{**}(x)$; а так как f замкнута и выпукла, $f^{**} = f$, поэтому $y \in \partial f(x)$.

Наконец, если f строго выпукла, то $f(z) - y^T z$ имеет единственную точку минимума по z , и эта точка есть $\nabla f^*(y)$.

Сильная выпуклость $\Rightarrow$ гладкость f^*

Пусть f μ -сильно выпукла. Обозначим $x_u = \nabla f^*(u)$, $x_v = \nabla f^*(v)$ — точки, в которых достигаются $\max_x [u^T x - f(x)]$ и $\max_x [v^T x - f(x)]$. Сильная выпуклость даёт два неравенства:

$$\begin{aligned} f(x_v) - u^T x_v &\geq f(x_u) - u^T x_u + \frac{\mu}{2} \|x_u - x_v\|^2 \\ f(x_u) - v^T x_u &\geq f(x_v) - v^T x_v + \frac{\mu}{2} \|x_u - x_v\|^2 \end{aligned}$$

Сильная выпуклость $\Rightarrow$ гладкость f^*

Пусть f μ -сильно выпукла. Обозначим $x_u = \nabla f^*(u)$, $x_v = \nabla f^*(v)$ — точки, в которых достигаются $\max_x [u^T x - f(x)]$ и $\max_x [v^T x - f(x)]$. Сильная выпуклость даёт два неравенства:

$$\begin{aligned} f(x_v) - u^T x_v &\geq f(x_u) - u^T x_u + \frac{\mu}{2} \|x_u - x_v\|^2 \\ f(x_u) - v^T x_u &\geq f(x_v) - v^T x_v + \frac{\mu}{2} \|x_u - x_v\|^2 \end{aligned}$$

Сложив, получаем $(u - v)^T (x_u - x_v) \geq \mu \|x_u - x_v\|^2$. Применив Коши—Буняковского—Шварца к левой части ($\leq \|u - v\| \|x_u - x_v\|$):

$$\|x_u - x_v\| \leq \frac{1}{\mu} \|u - v\|,$$

то есть ∇f^* липшицев с константой $1/\mu$.

Гладкость $f^* \Rightarrow$ сильная выпуклость f

Обозначим $g = f^*$, $L = \frac{1}{\mu}$. Так как ∇g липшицев с константой L , для $g_x(z) = g(z) - \nabla g(x)^T z$ верно

$$\frac{1}{2L} \|\nabla g(x) - \nabla g(y)\|^2 \leq g(y) - g(x) + \nabla g(x)^T (x - y).$$

Гладкость $f^* \Rightarrow$ сильная выпуклость f

Обозначим $g = f^*$, $L = \frac{1}{\mu}$. Так как ∇g липшицев с константой L , для $g_x(z) = g(z) - \nabla g(x)^T z$ верно

$$\frac{1}{2L} \|\nabla g(x) - \nabla g(y)\|^2 \leq g(y) - g(x) + \nabla g(x)^T (x - y).$$

Симметризуя по x, y и полагая $u = \nabla g(x)$, $v = \nabla g(y)$, получаем

$$(x - y)^T (u - v) \geq \frac{1}{L} \|u - v\|^2,$$

что и есть $\frac{1}{L} = \mu$ -сильная выпуклость f (через монотонность субградиента).

Галерея сопряжённых функций (справка)

$f(x)$	$f^*(y)$	где встречается
$\frac{1}{2}x^T Qx, \quad Q \succ 0$	$\frac{1}{2}y^T Q^{-1}y$	квадратичные задачи, двойственный подъём
$\frac{1}{2}\ x\ _2^2$	$\frac{1}{2}\ y\ _2^2$	ℓ_2 самосопряжена
$\ x\ $ (любая норма)	$I_{\{\ y\ _* \leq 1\}}$	$\ell_1 \rightarrow \ell_\infty$ -шар, LASSO

- **Квадратичная $\rightarrow$ квадратичная:** обусловленность инвертируется, но в двойственной задаче появляется ещё A , отсюда κ_d .

Галерея сопряжённых функций (справка)

$f(x)$	$f^*(y)$	где встречается
$\frac{1}{2}x^T Qx, \quad Q \succ 0$	$\frac{1}{2}y^T Q^{-1}y$	квадратичные задачи, двойственный подъём
$\frac{1}{2}\ x\ _2^2$	$\frac{1}{2}\ y\ _2^2$	ℓ_2 самосопряжена
$\ x\ $ (любая норма)	$I_{\{\ y\ _* \leq 1\}}$	$\ell_1 \rightarrow \ell_\infty$ -шар, LASSO

- **Квадратичная** $\rightarrow$ **квадратичная**: обусловленность инвертируется, но в двойственной задаче появляется ещё A , отсюда κ_d .
- **Норма** $\rightarrow$ **индикатор**: негладкая $\|x\|_1$ становится «жёстким» ограничением; это и использует ADMM, разделяя гладкую и негладкую части.

Свойства сопряжённой функции (справка)

- Связь с двойственной задачей:

$$-f^*(y) = \min_x [f(x) - y^T x]$$

Свойства сопряжённой функции (справка)

- Связь с двойственной задачей:

$$-f^*(y) = \min_x [f(x) - y^T x]$$

- Если f замкнута и выпукла, то $f^{**} = f$, а субдифференциалы взаимно обратны:

$$x \in \partial f^*(y) \Leftrightarrow y \in \partial f(x)$$

Свойства сопряжённой функции (справка)

- Связь с двойственной задачей:

$$-f^*(y) = \min_x [f(x) - y^T x]$$

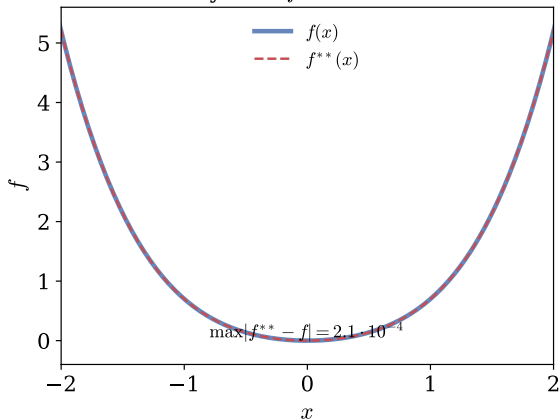
- Если f замкнута и выпукла, то $f^{**} = f$, а субдифференциалы взаимно обратны:

$$x \in \partial f^*(y) \Leftrightarrow y \in \partial f(x)$$

- Если f строго выпукла, то $\nabla f^*(y) = \arg \min_z [f(z) - y^T z]$ — единственная точка.

Второе сопряжённое f^{**}

Выпуклая $f = 0.5x^2 + 0.2x^4$



Невыпуклая $f = 0.25(x^2 - 1)^2$

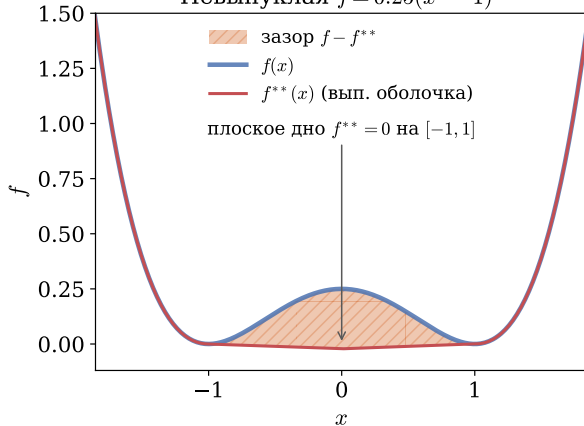


Figure 26: Бисопряжённая f^{**} : для выпуклой f совпадает с f , для невыпуклой даёт нижнюю выпуклую огибающую (зазор заштрихован).